

**LUNAVISION AI: AN INTELLIGENT MULTI-AGENT PIPELINE FOR  
AUTOMATED EXOPLANET DISCOVERY FROM TESS  
OBSERVATIONS**

**A PROJECT REPORT**

*Submitted by*

**PRASANNA B R (913123243042)**

**SANDEEP M (913123243047)**

**GOKUL S (913123243301)**

*In partial fulfillment for the award of the degree*

*of*

**BACHELOR OF TECHNOLOGY**

*in*

**ARTIFICIAL INTELLIGENCE AND DATA SCIENCE**



**VELAMMAL COLLEGE OF ENGINEERING AND TECHNOLOGY,  
MADURAI –625 009**

**(Autonomous)**

**OCTOBER 2026**

**VELAMMAL COLLEGE OF ENGINEERING AND TECHNOLOGY, MADURAI - 625009**  
**(AUTONOMOUS)**

**Department of Artificial Intelligence and Data Science**

**BONAFIDE CERTIFICATE**

This is to Certify that this project report “**LUNAVISION AI: AN INTELLIGENT MULTI-AGENT PIPELINE FOR AUTOMATED EXOPLANET DISCOVERY FROM TESS OBSERVATIONS**” is the bonafide work of “**PRASANNA B R (913123243042), SANDEEP M (913123243047), GOKUL S (913124243301)**” of **VII Semester** who carried out the project work under supervision.

**DR. S. SASIKALA**  
**HEAD OF THE DEPARTMENT**  
DEPARTMENT OF ARTIFICIAL  
INTELLIGENCE AND DATA SCIENCE  
VELAMMAL COLLEGE OF  
ENGINEERING AND TECHNOLOGY  
MADURAI

**MRS. M. BHAVADHARANI**  
**ASSISSTANT PROFESSOR**  
DEPARTMENT OF ARTIFICIAL  
INTELLIGENCE AND DATA SCIENCE  
VELAMMAL COLLEGE OF  
ENGINEERING AND TECHNOGY  
MADURAI

Submitted for the End Semester Viva Voce Examination held on \_\_\_\_\_ at  
Velammal College of Engineering and Technology, Madurai.

**INTERNAL EXAMINER**

**EXTERNAL EXAMINER**

## ACKNOWLEDGEMENT

We thank the almighty for giving us moral strength to work on the project for the past few months.

We would like to express our sincere thanks to **SHRI. M. V. MUTHURAMALINGAM**, Honorable Chairman, founder of the institution who had been a guiding light for us in all activities.

We express our sincere thanks to our Respected Principal, **Dr. P. ALLI**, for providing more facilities to do this project work.

Our heartfelt gratitude to **Dr. S. SASIKALA**, Professor and Head of the Department, Artificial Intelligence and Data Science Engineering, and our Project Guide for her valuable guidance, constant support, inspiration, and encouragement throughout the course of this project. Her insightful suggestions, constructive feedback, and continuous motivation greatly helped us in successfully completing the project within the stipulated time.

We convey our thanks to our Guide **Mrs. M. BHAVADHARANI**, Assistant Professor, Department of Artificial Intelligence and Data Science, for her innovative suggestions and valuable guidance. We would also wish to extend our sincere gratitude to all faculty members of the Department of Artificial Intelligence and Data Science for their valuable guidance throughout the course of our project. We also thank our parents and friends who provided moral and physical support.

# ABSTRACT

The discovery of exoplanets has become one of the most active areas of modern astronomy. NASA's Transiting Exoplanet Survey Satellite (TESS) records the brightness of tens of thousands of stars in every observing sector, and a planet crossing its host star appears as a small, periodic dip in the light curve. The volume and noise of these data make manual inspection impractical, and existing automated tools address only isolated stages of the workflow.

This project presents LuNaVision AI, an intelligent multi-agent pipeline for automated exoplanet discovery from TESS observations. Specialised agents handle data acquisition from NASA MAST, light-curve preprocessing and detrending, Transit Least Squares period detection, GPU-accelerated engineering of more than sixty physically motivated features, and classification with a leakage-safe stacked ensemble with calibrated probabilities. Parameter fitting, reporting and active learning are designed as further agents.

The pipeline was run on 2,360 real TESS light curves; 2,312 were detrended with a median noise reduction of about 49 percent. On a held-out set of 530 real targets the classifier achieves 74.2 percent accuracy, a ROC-AUC of 0.750 and a PR-AUC of 0.840, using interpretable transit-shape and signal-quality features. The report documents the design, implementation deviations, nine bugs found and fixed, benchmark studies and a path toward a SaaS product.

**Keywords:** Exoplanet detection, TESS, transit photometry, light curves, multi-agent systems, machine learning.

# TABLE OF CONTENTS

| CHAPTER NO. / TITLE                                      | PAGE NO.  |
|----------------------------------------------------------|-----------|
| <b>1. INTRODUCTION.....</b>                              | <b>1</b>  |
| 1.1 Background of the Study.....                         | 1         |
| 1.2 Motivation.....                                      | 2         |
| 1.3 Problem Statement.....                               | 3         |
| 1.4 Objectives.....                                      | 3         |
| 1.5 Scope of the Project.....                            | 4         |
| 1.6 Contributions of the Project.....                    | 5         |
| 1.7 Organisation of the Report.....                      | 5         |
| <b>2. LITERATURE SURVEY.....</b>                         | <b>6</b>  |
| 2.1 Overview of Exoplanet Detection Methods.....         | 6         |
| 2.2 Signal Processing for Weak Periodic Signals.....     | 6         |
| 2.3 Machine Learning for Light-Curve Classification..... | 7         |
| 2.4 Foundations Used by the Pipeline.....                | 8         |
| 2.5 Comparison of Surveyed Work.....                     | 8         |
| 2.6 Research Gap.....                                    | 9         |
| <b>3. SYSTEM ANALYSIS AND REQUIREMENTS.....</b>          | <b>10</b> |
| 3.1 Existing System.....                                 | 10        |
| 3.2 Proposed System.....                                 | 11        |
| 3.3 Functional Requirements.....                         | 12        |
| 3.4 Non-Functional Requirements.....                     | 13        |
| 3.5 Feasibility Study.....                               | 13        |
| 3.6 Hardware and Software Requirements.....              | 14        |
| <b>4. PROPOSED SYSTEM ARCHITECTURE.....</b>              | <b>15</b> |
| 4.1 Design Philosophy.....                               | 15        |
| 4.2 Agents and Their Responsibilities.....               | 16        |
| 4.3 Message Envelope.....                                | 17        |
| 4.4 End-to-End Data Flow.....                            | 18        |
| 4.5 Dynamic Behaviour of the Pipeline.....               | 18        |
| <b>5. DATASET AND DATA COLLECTION.....</b>               | <b>21</b> |

|                                                      |           |
|------------------------------------------------------|-----------|
| 5.1 Overview.....                                    | 21        |
| 5.2 Label and Catalogue Sources.....                 | 21        |
| 5.3 Unified Label Construction.....                  | 22        |
| 5.4 Light-Curve Acquisition.....                     | 23        |
| 5.5 Dataset Statistics.....                          | 24        |
| 5.6 Synthetic Transit Injection.....                 | 24        |
| 5.7 Data Quality Assessment.....                     | 25        |
| 5.8 Train, Validation and Test Split.....            | 25        |
| <b>6. METHODOLOGY AND IMPLEMENTATION.....</b>        | <b>26</b> |
| 6.1 Stage 1: Data Acquisition.....                   | 26        |
| 6.2 Stage 2: Preprocessing and Detrending.....       | 27        |
| 6.3 Stage 3: Periodicity Detection.....              | 29        |
| 6.4 Stage 4: Feature Engineering.....                | 30        |
| 6.5 Stage 5: Classification.....                     | 32        |
| 6.6 Leakage Control.....                             | 33        |
| 6.7 Web Dashboard and API.....                       | 34        |
| 6.8 Source Code Overview.....                        | 36        |
| <b>7. TESTING AND VALIDATION.....</b>                | <b>39</b> |
| 7.1 Testing Strategy.....                            | 39        |
| 7.2 Unit Testing.....                                | 39        |
| 7.3 Integration Testing.....                         | 40        |
| 7.4 Injection–Recovery Validation.....               | 40        |
| 7.5 Bug Log Summary.....                             | 41        |
| 7.6 Evaluation Protocol for the Classifier.....      | 41        |
| <b>8. RESULTS AND DISCUSSION.....</b>                | <b>42</b> |
| 8.1 Experimental Setup.....                          | 42        |
| 8.2 Acquisition and Catalogue Statistics (REAL)..... | 42        |
| 8.3 Preprocessing Performance.....                   | 43        |
| 8.4 Classification Performance (REAL).....           | 43        |
| 8.5 Discussion.....                                  | 47        |
| <b>9. COMPARATIVE ANALYSIS AND IMPACT.....</b>       | <b>48</b> |

|                                                          |           |
|----------------------------------------------------------|-----------|
| 9.1 Comparison of Development Strategies.....            | 48        |
| 9.2 Societal and Educational Impact.....                 | 48        |
| <b>10. RISKS, CONCLUSION AND FUTURE ENHANCEMENT.....</b> | <b>49</b> |
| 10.1 Risks and Mitigations.....                          | 49        |
| 10.2 Limitations.....                                    | 49        |
| 10.3 Conclusion.....                                     | 50        |
| 10.4 Future Enhancements.....                            | 51        |
| <b>APPENDIX A: FEATURE DICTIONARY.....</b>               | <b>53</b> |
| <b>APPENDIX B: COMMAND-LINE USAGE.....</b>               | <b>54</b> |
| <b>REFERENCES.....</b>                                   | <b>55</b> |

## LIST OF TABLES

|                                                                      |    |
|----------------------------------------------------------------------|----|
| Table 2.1: Capability comparison against surveyed literature.....    | 9  |
| Table 3.1: Functional requirements.....                              | 12 |
| Table 3.2: Non-functional requirements.....                          | 13 |
| Table 3.3: Hardware requirements.....                                | 14 |
| Table 3.4: Software requirements.....                                | 14 |
| Table 4.1: Agents and responsibilities.....                          | 16 |
| Table 4.2: Artefacts exchanged between stages.....                   | 18 |
| Table 5.1: Data sources.....                                         | 21 |
| Table 5.2: Disposition-to-label mapping.....                         | 22 |
| Table 5.3: Dataset snapshot (19 September 2026).....                 | 24 |
| Table 5.4: Injection parameter ranges.....                           | 24 |
| Table 6.1: Acquisition parameters.....                               | 26 |
| Table 6.2: Preprocessing parameters.....                             | 29 |
| Table 6.3: TLS configuration.....                                    | 29 |
| Table 6.4: Engineered feature groups.....                            | 30 |
| Table 6.5: Feature-extraction safeguards.....                        | 32 |
| Table 6.6: Base-learner hyper-parameters (HistGradientBoosting)..... | 33 |
| Table 7.1: Unit test cases.....                                      | 39 |
| Table 7.2: Defects found during testing.....                         | 41 |
| Table 8.1: Experimental setup.....                                   | 42 |
| Table 8.2: Pipeline data volumes by stage (REAL).....                | 43 |
| Table 8.3: Classifier metrics against project targets (REAL).....    | 44 |
| Table 8.4: Per-class results (REAL).....                             | 44 |
| Table 8.5: Top-10 permutation importances (REAL).....                | 46 |
| Table 9.1: Hand-chained scripts versus orchestrated pipeline.....    | 48 |
| Table 10.1: Identified risks and mitigations.....                    | 49 |
| Table A.1: Feature dictionary.....                                   | 53 |

## LIST OF FIGURES

|                                                                                                       |    |
|-------------------------------------------------------------------------------------------------------|----|
| Figure 3.1: Use case diagram of LuNaVision AI.....                                                    | 12 |
| Figure 4.1: Overall architecture of LuNaVision AI.....                                                | 15 |
| Figure 4.2: Class diagram of LuNaVision AI.....                                                       | 17 |
| Figure 4.3: Six-stage sequential pipeline (teal: implemented, grey: planned).....                     | 18 |
| Figure 4.4: Activity diagram with swimlanes for one pipeline run.....                                 | 19 |
| Figure 4.5: Sequence diagram of a successful pipeline run.....                                        | 20 |
| Figure 5.1: Dataset composition: class split of usable light curves and catalogue sizes<br>.....      | 22 |
| Figure 6.1: Effect of biweight detrending on a light curve with three transits<br>(illustrative)..... | 27 |
| Figure 6.2: Live analysis of TIC 316470945 with stage progress and calibrated score<br>.....          | 34 |
| Figure 6.3: Candidate explorer ranked by model score.....                                             | 35 |
| Figure 6.4: Target page with light curve, phase fold and measured features.....                       | 35 |
| Figure 8.1: Held-out confusion matrix (REAL).....                                                     | 44 |
| Figure 8.2: Learning curve on the held-out test set (REAL).....                                       | 45 |
| Figure 8.3: Top-12 features by permutation importance (REAL).....                                     | 46 |
| Figure 8.4: Model performance view of the dashboard.....                                              | 47 |

# LIST OF SYMBOLS, ABBREVIATIONS AND NOMENCLATURE

| Abbreviation / Symbol | Expansion / Meaning                                     |
|-----------------------|---------------------------------------------------------|
| <b>AI</b>             | Artificial Intelligence                                 |
| <b>BLS</b>            | Box Least Squares                                       |
| <b>CNN</b>            | Convolutional Neural Network                            |
| <b>CUDA</b>           | Compute Unified Device Architecture                     |
| <b>EB</b>             | Eclipsing Binary                                        |
| <b>BEB</b>            | Background Eclipsing Binary                             |
| <b>FAP</b>            | False Alarm Probability                                 |
| <b>FFI</b>            | Full-Frame Image                                        |
| <b>FFT</b>            | Fast Fourier Transform                                  |
| <b>FITS</b>           | Flexible Image Transport System                         |
| <b>GPU</b>            | Graphics Processing Unit                                |
| <b>MAST</b>           | Mikulski Archive for Space Telescopes                   |
| <b>MCMC</b>           | Markov Chain Monte Carlo                                |
| <b>PDCSAP</b>         | Pre-search Data Conditioning Simple Aperture Photometry |
| <b>PR-AUC</b>         | Precision-Recall Area Under the Curve                   |
| <b>ROC</b>            | Receiver Operating Characteristic                       |
| <b>SDE</b>            | Signal Detection Efficiency                             |
| <b>SMOTE</b>          | Synthetic Minority Over-sampling Technique              |
| <b>SNR</b>            | Signal-to-Noise Ratio                                   |
| <b>SPOC</b>           | Science Processing Operations Center                    |
| <b>TCE</b>            | Threshold-Crossing Event                                |
| <b>TESS</b>           | Transiting Exoplanet Survey Satellite                   |
| <b>TIC</b>            | TESS Input Catalog                                      |
| <b>TLS</b>            | Transit Least Squares                                   |
| <b>TOI</b>            | TESS Object of Interest                                 |
| <b>Rp/R*</b>          | Planet-to-star radius ratio                             |
| <b>P</b>              | Orbital period (days)                                   |
| <b>T0</b>             | Mid-transit epoch (BTJD)                                |
| <b>Teff</b>           | Effective temperature (K)                               |
| <b>log g</b>          | Surface gravity, log <sub>10</sub> (cm/s <sup>2</sup> ) |
| <b>ppm</b>            | Parts per million                                       |
| <b>σ</b>              | Standard deviation (sigma)                              |

# CHAPTER 1

## INTRODUCTION

### 1.1 Background of the Study

For most of human history the night sky was assumed to hold only one planetary system that we could ever know: our own. That assumption ended in 1995, when the first planet orbiting a Sun-like star was detected using precise radial-velocity measurements. Since then, the field of exoplanet science has grown from a curiosity into one of the most productive areas of modern astronomy. More than five thousand confirmed planets are now catalogued by the NASA Exoplanet Archive, and thousands more candidates await confirmation. These worlds range from scorching gas giants that orbit their stars in less than a day, to temperate rocky planets that might one day be examined for signs of atmospheres and, ultimately, life.

Exoplanets are discovered through several complementary techniques, including radial velocity, transit photometry, gravitational microlensing, astrometry and direct imaging. Of these, transit photometry has been the most prolific. When a planet passes in front of its host star as seen from Earth, it blocks a small fraction of the starlight and produces a brief, periodic dip in the star's measured brightness. For an Earth-sized planet crossing a Sun-like star the dip is only about 84 parts per million; for a Jupiter-sized planet it is roughly one percent. Detecting such tiny, repetitive dips requires very stable, continuous, high-precision photometry, which is best achieved from space.

NASA's Transiting Exoplanet Survey Satellite (TESS) [12], launched in April 2018, was designed exactly for this purpose. TESS carries four wide-

field cameras and observes a large strip of the sky, called a sector, for about 27 days at a time. Bright stars are recorded at a two-minute cadence, and the whole field is recorded in full-frame images at a much slower cadence. In every sector, tens of thousands of stars are monitored, and the mission has by now observed most of the sky. The Science Processing Operations Center (SPOC) pipeline converts the raw pixel data into calibrated light curves, in which the flux of each star is tabulated against time, and these products are archived at the Mikulski Archive for Space Telescopes (MAST).

The result is a data-rich but analyst-poor situation. Each sector adds another batch of light curves that must be screened for planet-like signals. A human analyst cannot visually inspect this volume of data, and even the automated vetting performed by the mission produces thousands of threshold-crossing events that must be examined further. Most of these events are not planets. They are eclipsing binary stars, background eclipsing binaries whose light is blended into the target aperture, stellar variability such as starspots and pulsations, and instrumental artefacts. Separating genuine planetary transits from these false positives is the central scientific and engineering problem addressed by this project.

## **1.2 Motivation**

Several observations motivated this work. First, the cost of manual vetting scales linearly with the number of candidates, whereas the number of observed stars keeps growing with every new sector. Second, the existing automated tools are narrow. A typical script downloads light curves, another cleans them, a third searches for periodic dips, and a classifier notebook expects a hand-prepared feature table as its input. Every hand-off between such tools is a place where formats drift, rows silently disappear and unlogged manual filtering enters the analysis. Third,

machine-learning classifiers reported in the literature frequently quote very high accuracy on benchmark datasets, but they seldom describe how the surrounding pipeline was built or how leakage between training and test data was prevented.

The team therefore chose to treat exoplanet discovery not as a single classification problem but as an end-to-end engineering problem: how can a raw TESS observation be carried through acquisition, cleaning, detection, feature extraction, classification, parameter estimation and reporting inside one coordinated, auditable system? The answer explored in this report is LuNaVision AI, a multi-agent pipeline in which each stage is a specialised software agent supervised by a central orchestrator.

### **1.3 Problem Statement**

To design and implement an automated, resumable and interpretable software pipeline that ingests raw NASA TESS light curves, removes instrumental and stellar noise, detects periodic transit-like dips, computes a rich set of physically motivated features, classifies each candidate against common false-positive classes, and produces auditable evidence for every decision, so that a new TESS sector can be processed with minimal manual intervention.

### **1.4 Objectives**

- To automate the acquisition of TESS two-minute cadence light curves from the MAST archive, with resumable, rate-limit-aware downloading and quality-flag filtering.
- To build a preprocessing stage that normalises, cleans and detrends light curves without destroying the transit signal that must later be detected.

- To detect periodic transit signals with the Transit Least Squares (TLS) algorithm and to convert the search output into calibrated signal-quality metrics such as the Signal Detection Efficiency (SDE) and false-alarm probability.
- To engineer a feature vector of more than sixty physically meaningful attributes, covering transit geometry, odd/even depth consistency, secondary eclipses, phase-folded morphology, a centroid-shift proxy, stellar parameters and spectral statistics, using GPU acceleration where hardware allows.
- To train a leakage-safe stacked ensemble classifier that outputs calibrated transit probabilities, and to report its performance honestly against the project targets.
- To document the design so that further stages, namely Bayesian parameter fitting, automated reporting, active learning and a web dashboard, can be plugged into the same architecture.

## 1.5 Scope of the Project

The scope of the present work covers the first five pipeline stages as working software (data acquisition, preprocessing, periodicity detection, feature engineering and classification) and a dashboard prototype. Bayesian parameter fitting, automated reporting and the active-learning loop are specified at design level and described as future enhancements. The data are TESS Sector 1 two-minute cadence light curves for a label-driven sample of targets, cross-matched with catalogues from ExoFOP-TESS, the NASA Exoplanet Archive and the SPOC threshold-crossing-event list. The classifier is evaluated as a binary Transit versus Other problem; the four-class taxonomy is the next milestone.

## 1.6 Contributions of the Project

- A modular, resumable end-to-end pipeline whose stages communicate through typed messages and on-disk artefacts, enabling fault isolation and independent scaling.
- A label-driven acquisition strategy that interleaves classes so that any prefix of the download is balanced, thereby making partial runs scientifically useful.
- A GPU-accelerated feature extractor with automatic CPU fallback, worker caps, per-task timeouts, RAM guards and batch checkpointing.
- A group-aware evaluation protocol that prevents a synthetic light curve from sharing a fold with its host star, and explicit removal of two leaky features that inflated importance scores.
- A documented bug log, in which nine concrete root causes were found and fixed, offering a practical lesson for anyone building similar pipelines.

## 1.7 Organisation of the Report

Chapter 2 reviews the literature and identifies the research gap. Chapters 3 and 4 present the system analysis and the multi-agent architecture. Chapter 5 documents the datasets and Chapter 6 the implementation of each stage. Chapter 7 describes testing, Chapter 8 presents the results, and Chapter 9 gives the comparative analysis and impact. Chapter 10 covers risks, the conclusion and future work. The appendices give a feature dictionary, and command-line usage.

## **CHAPTER 2**

### **LITERATURE SURVEY**

#### **2.1 Overview of Exoplanet Detection Methods**

Wright and Gaudi [8] review the principal methods by which exoplanets are found. Radial-velocity surveys measure the periodic Doppler shift of stellar spectral lines caused by an orbiting planet. Transit photometry measures the periodic dimming caused by a planet crossing the stellar disc. Direct imaging suppresses starlight and photographs the planet itself, while microlensing exploits the gravitational amplification of a background star. Each method has different biases: radial velocity favours massive close-in planets around quiet stars, transits favour close-in planets whose orbits happen to be aligned with the line of sight, and direct imaging favours young, hot giants on wide orbits.

Transit photometry offers a unique advantage for automated processing. The observable is a single, uniformly sampled time series per star, which can be handled with standard signal-processing and machine-learning tools, and the geometry of the transit directly yields the planet-to-star radius ratio, the orbital period and, given stellar parameters, the planetary radius. This is why missions such as Kepler, K2, CoRoT and TESS all adopted the transit method.

#### **2.2 Signal Processing for Weak Periodic Signals**

Khan, Jenkins and Yoma [7] survey signal-processing techniques for radial-velocity exoplanet detection and show how statistical thresholds and filtering improve the recovery of weak periodic signals buried in noise. Although their focus is radial velocity rather than transit photometry, the principle carries over: a detection statistic must be defined, its null

distribution understood, and a threshold chosen to trade completeness against false alarms. In transit searches this role is played by the Signal Detection Efficiency and the false-alarm probability. De Beurs and colleagues [6] use neural networks to remove stellar activity signals from radial-velocity data before a planet search is performed, illustrating that stellar noise must be modelled, not merely ignored.

### **2.3 Machine Learning for Light-Curve Classification**

Jara-Maldonado and co-authors [2] apply K-nearest neighbours in combination with the SMOTE oversampling technique to stellar light-intensity data and obtain high accuracy while correcting for the scarcity of positive examples. This paper is representative of a whole family of classical classifiers, including logistic regression, support vector machines and random forests, which work well when the input is a fixed-length numerical vector. Pimentel, Amorim and Rudzicz [3] show that feature extraction drawing on several feature domains, such as time-domain statistics, frequency-domain descriptors and geometric transit parameters, improves classification over any single domain, and they use cross-validation explicitly to control for dataset bias.

Ansdell et al. [4] incorporated scientific domain knowledge, in particular centroid time series, into a deep convolutional network trained on Kepler data. Adding centroid information improved accuracy and recall, and helped detect low-signal-to-noise transits, because a shifting centroid during a dip reveals that the dimming source is not the target star but a nearby contaminant. Priyadarshini and Puri [5] describe a CNN-based ensemble that reports an accuracy of 99.62% on its benchmark set, the highest single figure among the surveyed works. The high figure should nevertheless be interpreted with care because the benchmark is far cleaner and better balanced than a real survey sector.

Feng and colleagues [1] apply differentiable rendering to direct-imaging data, modelling the imaging optics explicitly and thereby improving contrast and faint-planet recovery. Their work shows how physical forward models can complement purely statistical learners, an idea that motivates the transit-model fitting stage in the present project.

## **2.4 Foundations Used by the Pipeline**

Beyond the eight surveyed papers, the pipeline builds directly on established open-source detection and detrending methods. Hippke and Heller [9] introduced the Transit Least Squares algorithm, which replaces the simple box of the older Box Least Squares (BLS) method by a physically realistic transit shape including limb darkening and ingress/egress. TLS is more sensitive to small planets and yields a better-behaved detection statistic, at the cost of longer computation. Hippke and co-authors [10] present Wotan, a collection of time-series detrending filters, among which the sliding biweight filter is shown to preserve transit depth and shape better than polynomial or spline detrending while still removing stellar variability. Finally, the Lightkurve package [11] provides community-maintained tools to query, download and manipulate Kepler and TESS light curves, and is used for data access in this project.

## **2.5 Comparison of Surveyed Work**

Table 2.1 summarises the capability of each surveyed work along the dimensions that matter for an automated survey pipeline: whether it covers the whole workflow, whether it uses multiple cooperating agents, whether its decisions can be explained, whether it estimates physical parameters and whether it exploits GPU hardware.

**Table 2.1: Capability comparison against surveyed literature**

| Reference                    | End-to-end | Multi-agent | Explainable | Parameter est. | GPU |
|------------------------------|------------|-------------|-------------|----------------|-----|
| [1] Differentiable rendering | No         | No          | Partial     | No             | No  |
| [2] KNN + SMOTE              | No         | No          | No          | No             | No  |
| [3] Multi-domain features    | No         | No          | Partial     | No             | No  |
| [4] Domain-informed CNN      | No         | No          | No          | No             | No  |
| [5] CNN-based ensemble       | No         | No          | No          | No             | No  |
| [6] DL for RV activity       | No         | No          | No          | No             | No  |
| [7] RV signal processing     | No         | No          | No          | No             | No  |
| [8] Methods review           | N/A        | No          | N/A         | N/A            | N/A |
| LuNaVision AI (this work)    | Yes        | Yes         | Partial     | Planned        | Yes |

## 2.6 Research Gap

Each surveyed work is strong within its own stage but stops there. References [2] and [5] report only classification metrics and assume an already-detected, already-labelled candidate. References [3] and [4] depend on hand-engineered or side-channel inputs that must be supplied separately. Reference [1] requires wavefront data that a transit survey does not have, and the radial-velocity work [6], [7] does not transfer directly to space-based transit photometry. None of the surveyed papers implements and evaluates acquisition, preprocessing, detection, classification, parameter estimation, explanation and reporting as one coupled system; each addresses at most two of these stages, with a human operator connecting the rest.

The gap that this project closes is therefore architectural: a single orchestrated pipeline that carries a raw TESS observation through every stage without a human in the loop, while remaining interpretable enough for a domain scientist to audit any decision. The project also pays attention to leakage-safe evaluation, which the surveyed papers rarely discuss.

## **CHAPTER 3**

### **SYSTEM ANALYSIS AND REQUIREMENTS**

#### **3.1 Existing System**

In current practice, candidate vetting is a mixture of mission pipelines and community scripts. The SPOC pipeline produces light curves and data validation reports for threshold-crossing events, and the TESS Science Office publishes the catalogue of TESS Objects of Interest (TOIs). Independent researchers then re-download light curves, write their own detrending and search scripts and train classifiers on hand-assembled feature tables, so every group reimplements the same chain with its own conventions, undocumented filtering rules and evaluation split.

##### ***3.1.1 Limitations of the Existing System***

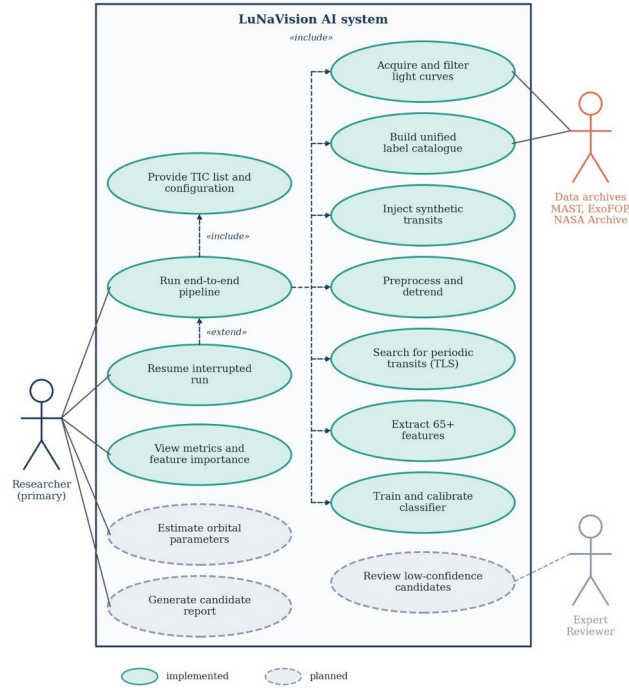
- Fragmentation: acquisition, detrending, search and classification are separate programs with no shared interface.
- Manual hand-offs: outputs are copied, reformatted and filtered by hand, introducing silent errors that are invisible to later reviewers.
- Poor resumability: an interrupted multi-hour download or feature-extraction run often has to restart from the beginning.
- Limited interpretability: classifiers are reported as black-box scores, without the feature-level evidence a scientist needs.
- Evaluation leakage: synthetic augmentations and duplicate targets frequently straddle training and test folds, inflating the reported accuracy.

### 3.2 Proposed System

The proposed LuNaVision AI system replaces the chain of hand-operated scripts with one orchestrated pipeline. Each stage is an independent Python module with a file-based interface, wrapped by a thin agent layer that exchanges typed messages. The orchestrator dispatches stages, retries on failure and records state, and long-running stages skip finished work so a run can be resumed. The classifier outputs calibrated probabilities with feature importances, and the evaluation protocol prevents leakage.

Figure 3.1 presents the use case diagram of the proposed system. The researcher is the primary actor: they supply the TIC list and configuration, start the end-to-end pipeline, resume an interrupted run and inspect the metrics and feature importances. The public archives (MAST, ExoFOP and the NASA Exoplanet Archive) act as external data providers for light-curve acquisition and label construction. Use cases drawn with a dashed outline, such as orbital-parameter estimation, report generation and expert review of low-confidence candidates, are planned and are not part of the current implementation.

Use case diagram of LuNaVision AI



**Figure 3.1: Use case diagram of LuNaVision AI**

### 3.2.1 Advantages of the Proposed System

- End-to-end automation from a target list to a scored candidate table.
- Fault isolation and independent scaling through loosely coupled stages.
- Interpretability through calibrated probabilities and feature-importance ranking.

## 3.3 Functional Requirements

**Table 3.1: Functional requirements**

| ID   | Requirement                                                                                                               | Stage       |
|------|---------------------------------------------------------------------------------------------------------------------------|-------------|
| FR-1 | Query MAST for TESS 2-minute SPOC light curves for a list of TIC identifiers and download them concurrently with retries. | Acquisition |
| FR-2 | Filter cadences using the TESS QUALITY bitmask and store TIME, PDCSAP_FLUX and flags.                                     | Acquisition |
| FR-3 | Merge ExoFOP, NASA Exoplanet Archive and SPOC catalogues into a unified label table.                                      | Acquisition |

| ID    | Requirement                                                                                                    | Stage          |
|-------|----------------------------------------------------------------------------------------------------------------|----------------|
| FR-4  | Inject synthetic transits into quiet light curves for class augmentation.                                      | Acquisition    |
| FR-5  | Normalise, clip, gap-fill and detrend each light curve, and reject unusable ones.                              | Preprocessing  |
| FR-6  | Run a TLS period search and compute SDE, FAP, depth, duration and period.                                      | Detection      |
| FR-7  | Compute 65+ features per candidate, using GPU where available.                                                 | Features       |
| FR-8  | Train, calibrate and persist a stacked classifier; report accuracy, macro-F1, ROC-AUC, PR-AUC and Brier score. | Classification |
| FR-9  | Estimate orbital parameters with uncertainties (planned).                                                      | Fitting        |
| FR-10 | Produce diagnostic plots and a ranked candidate report (planned).                                              | Reporting      |

## 3.4 Non-Functional Requirements

**Table 3.2: Non-functional requirements**

| Property         | Requirement                                                  | How it is met                                                       |
|------------------|--------------------------------------------------------------|---------------------------------------------------------------------|
| Scalability      | Handle one target or a whole sector with the same code path. | Directory-based batch functions; joblib parallelism.                |
| Resumability     | Continue after interruption without recomputation.           | Skip-if-exists checks; JSON manifests; checkpointing every 10 rows. |
| Robustness       | A hung or failing target must not stall the run.             | Per-task timeout (10 min); RAM guard; capped workers.               |
| Interpretability | Explain which properties drive a decision.                   | Calibrated probabilities; permutation importance.                   |
| Extensibility    | Swap detectors or classifiers without touching other stages. | Narrow module interfaces; optional XGBoost/LightGBM backends.       |
| Performance      | Full sector in at most 90 minutes on 8 CPU cores.            | Vectorised folding, 10-minute binning for period search, GPU FFT.   |

## 3.5 Feasibility Study

### 3.5.1 Technical Feasibility

All required algorithms are available as mature open-source Python packages: astroquery and Lightcurve for data access, Wotan for detrending, TLS for period search, batman for transit modelling, scikit-

learn for the ensemble, and CuPy and PyTorch for GPU kernels. The hardware in Table 3.3 proved sufficient for thousands of light curves.

### 3.5.2 *Economic Feasibility*

The project uses only free public data and open-source software. The dominant cost is time rather than money: MAST throughput limited downloads to roughly three to six targets per minute.

### 3.5.3 *Operational Feasibility*

The pipeline is run from the command line and writes plain CSV, NPZ and JSON files that any astronomer can open; a dashboard prototype gives a graphical view for demonstration.

## 3.6 Hardware and Software Requirements

**Table 3.3: Hardware requirements**

| Component      | Minimum                      | Used in this project                  |
|----------------|------------------------------|---------------------------------------|
| Processor      | 4-core CPU                   | macOS and Windows laptop CPUs         |
| Memory         | 8 GB RAM (1.5 GB free guard) | 16 GB                                 |
| GPU (optional) | CUDA-capable, 4 GB           | NVIDIA RTX 4050 Laptop GPU, CUDA 12.9 |
| Storage        | 20 GB free                   | NPZ light curves and CSV catalogues   |
| Network        | Broadband                    | MAST download (3 to 6 targets/min)    |

**Table 3.4: Software requirements**

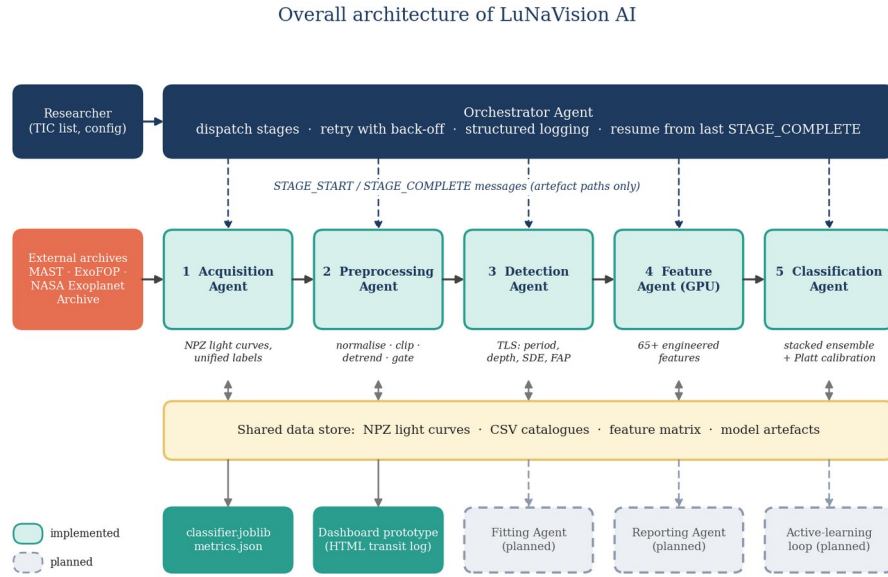
| Category               | Software                                                        |
|------------------------|-----------------------------------------------------------------|
| Language               | Python 3.9 and above                                            |
| Data access            | astroquery, astropy, lightkurve                                 |
| Detrending and search  | wotan, transitleastsquares                                      |
| Numerics and utilities | numpy, scipy, pandas, numba, joblib, psutil, tqdm, matplotlib   |
| GPU                    | CuPy (CUDA 11.x wheel), PyTorch                                 |
| Machine learning       | scikit-learn, XGBoost, LightGBM, imbalanced-learn, SHAP, Optuna |

# CHAPTER 4

## PROPOSED SYSTEM ARCHITECTURE

### 4.1 Design Philosophy

LuNaVision AI is organised as a multi-agent pipeline coordinated by a central Orchestrator Agent, as shown in Figure 4.1. The orchestrator dispatches each stage in sequence, retries a stage on failure and logs the state of every run so that the pipeline can be resumed after an interruption. Seven specialised agents sit beneath it, and an active-learning loop routes low-confidence candidates to an expert reviewer and folds the new labels into the next training run.



**Figure 4.1: Overall architecture of LuNaVision AI**

The message-passing design was chosen over a monolithic script for three reasons. The first is fault isolation: a failure in one agent, such as a MAST rate-limit error during acquisition, does not corrupt the agents downstream, because each agent consumes only the last successfully published output of its predecessor. The second is independent scaling: feature engineering is the most expensive stage and the only one that benefits from a GPU, so

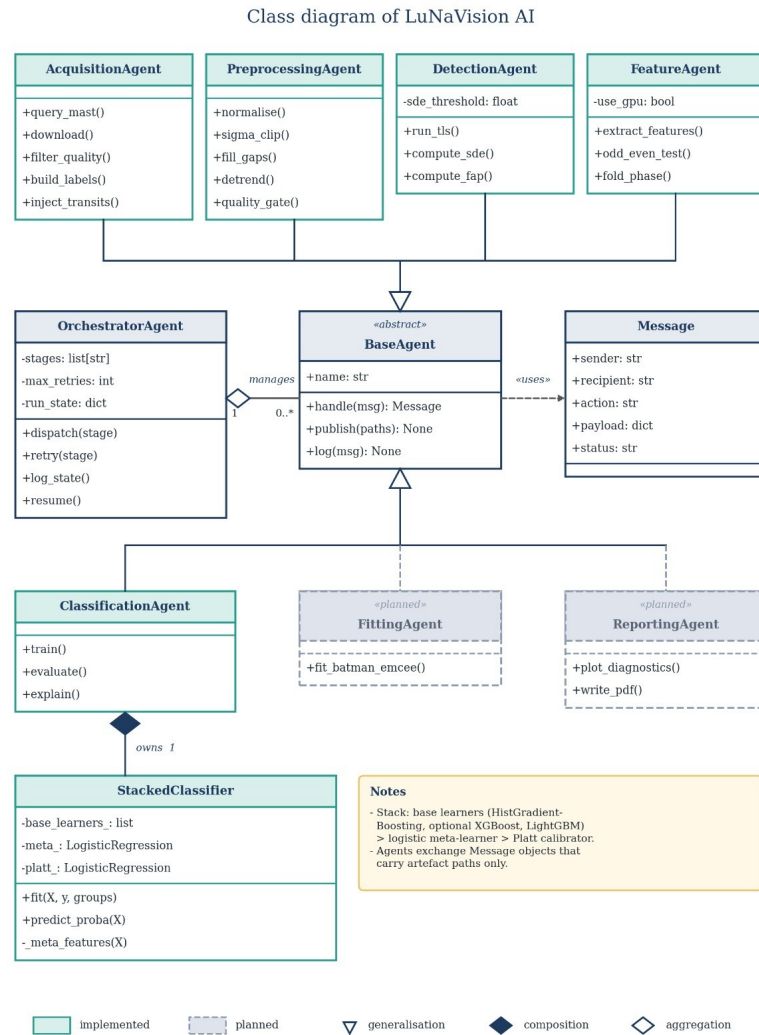
it can run on GPU hardware while lighter agents use CPU workers. The third is auditability: every message crossing an agent boundary is logged, so the intermediate state behind any classification can be reconstructed.

## 4.2 Agents and Their Responsibilities

**Table 4.1: Agents and responsibilities**

| Agent               | Module                       | Responsibility                                                                             | Status      |
|---------------------|------------------------------|--------------------------------------------------------------------------------------------|-------------|
| Orchestrator        | agents/orchestrator.py       | Dispatch stages, retry, log run state, resume                                              | Designed    |
| Data Acquisition    | acquisition.py               | Query MAST, download PDCSAP light curves, build label catalogue, inject synthetic transits | Implemented |
| Preprocessing       | preprocessing.py             | Normalise, sigma-clip, fill gaps, detrend, quality-gate                                    | Implemented |
| Transit Detection   | detection.py                 | TLS period search, SDE and FAP, candidate thresholds                                       | Implemented |
| Feature Engineering | features.py, gpu_features.py | Compute 65+ features with GPU acceleration                                                 | Implemented |
| Classification      | classifier.py                | Stacked ensemble with Platt calibration; evaluation and explanation                        | Implemented |
| Parameter Fitting   | fitting.py                   | batman + emcee posterior estimation                                                        | Planned     |
| Reporting           | report.py, visualization.py  | Diagnostic plots and PDF summary                                                           | Planned     |

The static structure of the implementation is summarised by the class diagram in Figure 4.2. Every agent specialises the abstract BaseAgent, which provides message handling, artefact publishing and logging. The OrchestratorAgent aggregates the agents and owns the run state used for retries and resumption, while the ClassificationAgent owns the StackedClassifier. Messages exchanged between agents are instances of the Message class and carry artefact paths only. Classes with a dashed outline (FittingAgent and ReportingAgent) are planned.



**Figure 4.2: Class diagram of LuNaVision AI**

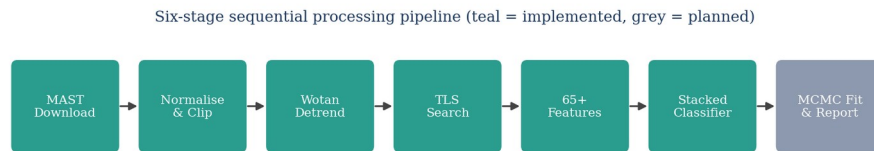
### 4.3 Message Envelope

Agents communicate with the orchestrator through a small, typed message envelope with five fields: sender, recipient, action, payload and status. The payload never carries bulk data, only the paths of the artefacts a stage produced, such as the directory of detrended NPZ files or the feature-matrix CSV. This keeps messages small and easy to log, and lets a stage be re-run in isolation by re-sending its input message.

```
{ "sender": "preprocessing_agent", "recipient": "orchestrator",
  "action": "STAGE_COMPLETE", "status": "ok",
  "payload": { "output_dir": "data/processed/lc_detrended",
               "n_ok": 2312, "n_rejected": 247 },
}
```

## 4.4 End-to-End Data Flow

Figure 4.3 shows the six processing stages and the artefacts passed between them. Raw light curves are stored as compressed NumPy archives and preprocessing writes detrended versions to a separate directory. The detection and feature stages read those files with the TIC stellar-parameter catalogue and write one feature matrix, and the classifier reads that matrix and the synthetic-injection table and writes a serialised model and a metrics file.



**Figure 4.3: Six-stage sequential pipeline (teal: implemented, grey: planned)**

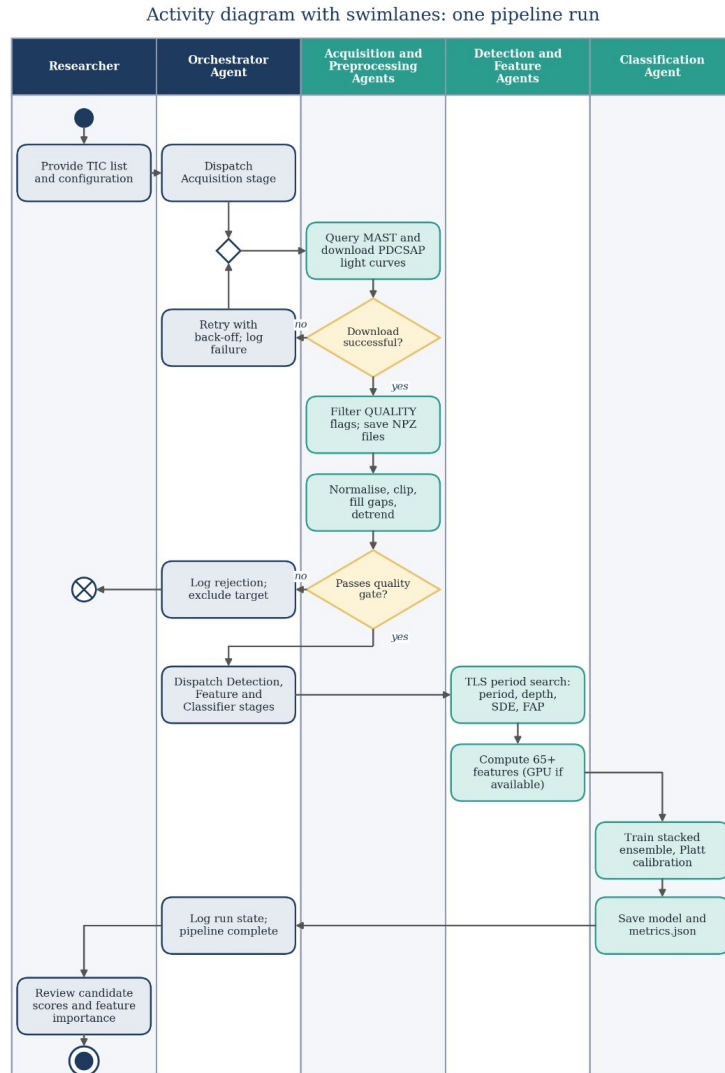
**Table 4.2: Artefacts exchanged between stages**

| Stage               | Input                                           | Output                                                                             |
|---------------------|-------------------------------------------------|------------------------------------------------------------------------------------|
| Acquisition         | TIC list, catalogues                            | data/processed/lc_raw/*.npz;<br>data/catalogs/unified_labels.csv;<br>manifest JSON |
| Preprocessing       | lc_raw/*.npz                                    | lc_detrended/*.npz;<br>preprocessing_report.csv                                    |
| Detection           | lc_detrended/*.npz                              | candidate table (period, depth, duration,<br>SDE, FAP)                             |
| Feature engineering | lc_detrended/*.npz;<br>tic_stellar_params.csv   | data/catalogs/feature_matrix.csv                                                   |
| Classification      | feature_matrix.csv;<br>synthetic_injections.csv | models/classifier.joblib;<br>models/metrics.json                                   |

## 4.5 Dynamic Behaviour of the Pipeline

The activity diagram in Figure 4.4 follows one pipeline run across the roles that take part in it. The Orchestrator dispatches each stage and retries a failed download with back-off; a light curve that fails the quality gate is

logged and excluded, and the remaining stages proceed only for targets that pass. The final activity saves the trained model and metrics.json for the researcher to review.



**Figure 4.4: Activity diagram with swimlanes for one pipeline run**

Figure 4.5 shows the same run as a sequence of messages. Each stage reports `STAGE_COMPLETE` to the Orchestrator with the artefact it produced, and every message is logged so that an interrupted run can resume from the last completed stage.

Sequence diagram of a successful pipeline run

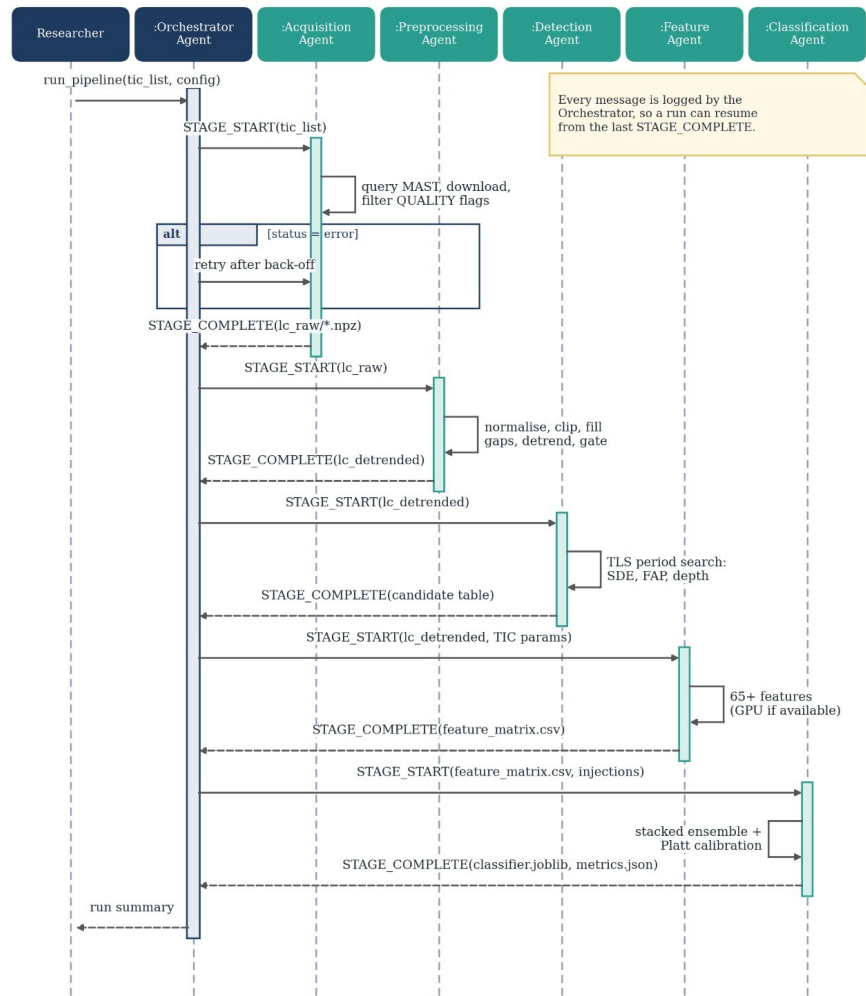


Figure 4.5: Sequence diagram of a successful pipeline run

# CHAPTER 5

## DATASET AND DATA COLLECTION

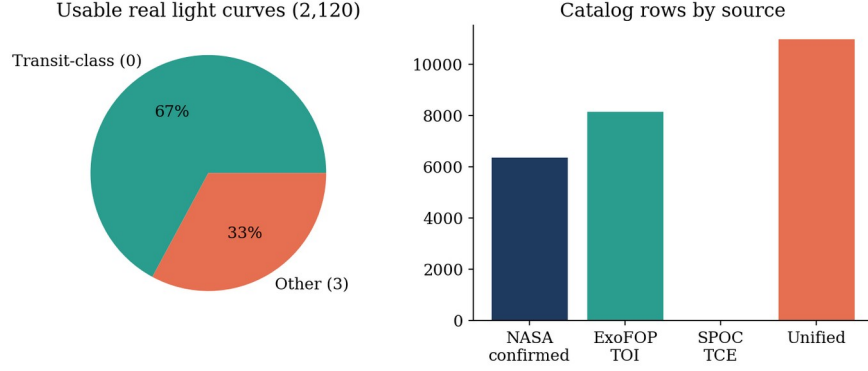
### 5.1 Overview

The quality of the training data determines the quality of any supervised classifier. Because no single archive provides both TESS light curves and consistent vetting labels, the project assembles its dataset from several complementary sources: catalogues that supply labels and physical parameters, the light curves themselves from MAST, and synthetic transits that supplement the scarce positive examples. This chapter describes each source, the construction of the unified label set and the quality checks applied.

### 5.2 Label and Catalogue Sources

**Table 5.1: Data sources**

| Source                    | Content                                                 | Rows (this project)            | Role                 |
|---------------------------|---------------------------------------------------------|--------------------------------|----------------------|
| ExoFOP-TESS TOI catalogue | TOIs with TFOPWG dispositions (CP, KP, PC, APC, FP, FA) | 8,149                          | Primary labels       |
| NASA Exoplanet Archive    | Confirmed planets with host-star parameters             | 6,367                          | Positive labels      |
| TESS SPOC TCE (Sector 1)  | Threshold-crossing events                               | 4 (sample)                     | Feature pre-training |
| TIC stellar parameters    | Teff, log g, radius, mass, distance, metallicity        | 501 (100 columns)              | Stellar features     |
| Synthetic injections      | batman-modelled transits in real light curves           | 199 generated (192 usable)     | Class augmentation   |
| Unified labels            | Merged priority-ordered label set                       | 10,965 (10,964 unique targets) | Label lookup         |



**Figure 5.1: Dataset composition: class split of usable light curves and catalogue sizes**

### 5.3 Unified Label Construction

Each catalogue expresses dispositions in its own vocabulary. The acquisition stage maps them onto one taxonomy. Planet candidates, known planets, confirmed planets, ambiguous planet candidates and verified planets are labelled as Transit (class 0). Eclipsing binaries are class 1, background eclipsing binaries class 2, and false positives, false alarms and variable stars are class 3. When a target appears in more than one catalogue, a priority order resolves the conflict: the NASA confirmed-planet table overrides the ExoFOP TOI table, which overrides the SPOC TCE table. This produces one label per target.

**Table 5.2: Disposition-to-label mapping**

| Raw disposition     | Unified label | Class name             |
|---------------------|---------------|------------------------|
| PC, KP, CP, APC, VP | 0             | Transit                |
| EB                  | 1             | Eclipsing binary       |
| BEB                 | 2             | Blend                  |
| FP, FA, V           | 3             | Other / false positive |

In the current data, only classes 0 and 3 are populated in sufficient numbers. Eclipsing-binary and blend labels are absent from the sample of targets that actually have SPOC two-minute light curves, which is why the classifier is presently evaluated as a binary problem.

## 5.4 Light-Curve Acquisition

### 5.4.1 *Label-driven sampling*

A whole TESS sector contains tens of thousands of stars, but only a small fraction has a label. The team therefore switched from sector-wise downloading to a label-driven strategy: the target list is drawn from the labelled catalogue, at most two sectors are fetched per target, and classes are interleaved so that any prefix of the download is class-balanced. If a run is stopped early, the partial data are still usable for training.

### 5.4.2 *Robust downloading*

MAST is a shared public service, and the throughput available to a single client is limited. The downloader therefore uses a thread pool with a concurrency of eight, staggered start times, exponential back-off with random jitter, explicit handling of HTTP 429 (too many requests) responses, and detection and repair of a corrupted Lightcurve cache. A manifest JSON records the outcome for every target, and files that already exist are skipped on the next run.

### 5.4.3 *Quality-flag filtering*

From each FITS product the pipeline extracts TIME, PDCSAP\_FLUX, SAP\_FLUX and QUALITY. Cadences whose QUALITY value has any of the bits 1, 2, 4, 8, 16 or 512 set are removed, because these flag attitude tweaks, safe modes, coarse pointing, Earth-point pointing, argabrightening and impulsive outliers respectively. The result is stored in an NPZ archive per target.

## 5.5 Dataset Statistics

**Table 5.3: Dataset snapshot (19 September 2026)**

| Quantity                                                            | Value                                 |
|---------------------------------------------------------------------|---------------------------------------|
| Targets requested                                                   | about 5,000                           |
| Real light curves downloaded                                        | 2,360                                 |
| Usable real light curves                                            | 2,120 (1,423 label 0 and 697 label 3) |
| Synthetic light curves                                              | 199 (192 usable)                      |
| Unique targets in unified label catalogue                           | 10,964                                |
| Downloads failing (mostly FFI-only targets without SPOC 2-min data) | about 28%                             |
| Effective download throughput                                       | 3 to 6 targets per minute             |

## 5.6 Synthetic Transit Injection

Confirmed planets are rare compared with non-planets, and the minority class in this pipeline is scientifically the interesting one. To boost it, transits are injected into real, quiet light curves using the batman transit model [13]. Parameters are drawn for the planet-to-star radius ratio, the orbital period and the inclination, and the injected star is labelled as a transit host.

A first version of the injector drew parameters uniformly and produced many scientifically invalid injections: some had an impact parameter so large that the planet never crossed the star, so the injected light curve was identical to its host, and others gave fewer than three transits or were buried in noise. The injector was therefore rewritten to use rejection sampling: a candidate is accepted only if the batman model shows a real dip, at least three transits fall inside the baseline, the injected signal-to-noise ratio is at least eight, and the host is a label-3 (non-planet) star.

**Table 5.4: Injection parameter ranges**

| Parameter              | Range        | Constraint                 |
|------------------------|--------------|----------------------------|
| Radius ratio $R_p/R_*$ | 0.01 to 0.15 | real dip verified by model |

| Parameter    | Range                | Constraint                           |
|--------------|----------------------|--------------------------------------|
| Period       | 1 to 30 days         | not longer than baseline / 3         |
| Inclination  | 85 to 90 degrees     | impact parameter below $1 + R_p/R^*$ |
| Injected SNR | at least 8           | relative to host noise               |
| Host star    | label 3 (non-planet) | avoids ambiguous labels              |

## 5.7 Data Quality Assessment

A separate analysis of the ten CSV files in the project examined missing values, duplicates, identifier consistency and leakage risks. The main findings were as follows.

- Label encoding: the label column uses the values 0 and 3 rather than 0 and 1, which must be handled explicitly by any downstream code.
- Missing stellar parameters: stellar mass is missing for about four percent of ExoFOP TOI rows and log g for about 4.4 percent; the classifier handles missing values natively through histogram-based gradient boosting.
- Identifier formats: light-curve files use the string form TIC\_123 while catalogues use the integer 123; a normalisation function was added after this mismatch turned most real-target labels into -1 (Bug 3 in the appendix).

## 5.8 Train, Validation and Test Split

Random row-wise splitting is inappropriate because a synthetic light curve shares its host star with real targets. The project therefore uses group-aware stratified splitting: every row belongs to a group (the target itself, or the host star for synthetic rows) and all rows of a group stay on the same side of any split. Synthetic rows are used for training only. The current split has 1,741 training rows and 530 held-out real test rows (356 Transit and 174 Other).

## CHAPTER 6

### METHODOLOGY AND IMPLEMENTATION

This chapter describes how each pipeline stage is implemented, the parameters chosen, and, importantly, the places where the implementation deliberately departs from the original specification because testing against real MAST data showed the specification to be wrong or impractical.

#### 6.1 Stage 1: Data Acquisition

The acquisition module (`acquisition.py`, the largest module in the project) queries MAST through `astroquery`, downloads TESS PDCSAP light curves through `Lightkurve`, parses FITS data with `astropy`, and builds the label catalogue. It exposes a command-line interface with a labelled-target mode, a per-target sector limit and a target-count limit. Downloads are executed by a thread pool. Each worker waits a small random delay before starting, so that the archive is not hit by all workers simultaneously, and retries failed requests with exponential back-off and jitter.

Two speed-ups were added after profiling. Phase folding, used repeatedly in the injection and feature code, was rewritten with `numpy` `bincount` and became about six times faster on CPU with bit-identical output. Gap detection was vectorised and became about forty-one times faster.

**Table 6.1: Acquisition parameters**

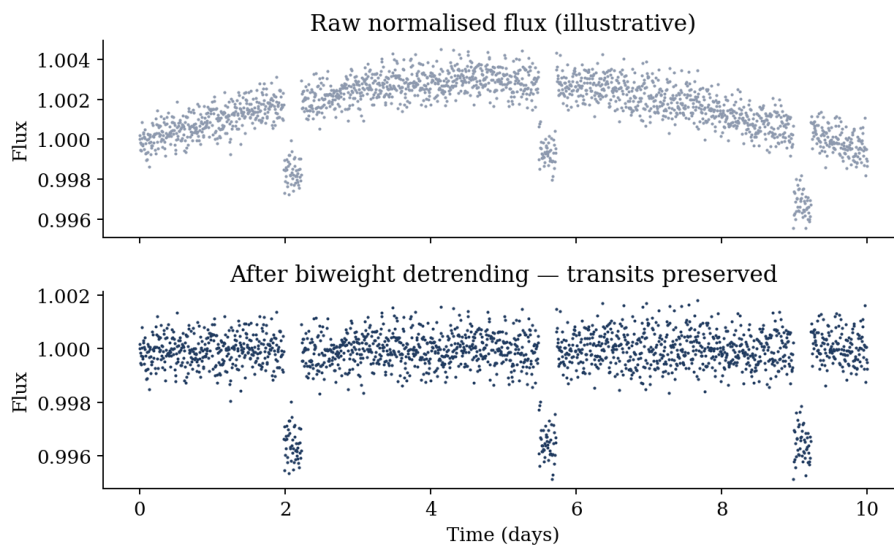
| Parameter                 | Value                                       |
|---------------------------|---------------------------------------------|
| Concurrency (thread pool) | 8 workers with staggered starts             |
| Retry policy              | exponential back-off with jitter; 429-aware |
| Cadence and author        | 2-minute SPOC                               |
| Flux used                 | PDCSAP_FLUX (SAP_FLUX also stored)          |
| QUALITY mask              | bits 1, 2, 4, 8, 16, 512 must be zero       |
| Sectors per target        | at most 2 in labelled mode                  |

| Parameter  | Value                                                         |
|------------|---------------------------------------------------------------|
| Skip logic | existing NPZ files and manifest entries are not re-downloaded |

## 6.2 Stage 2: Preprocessing and Detrending

The preprocessing module implements a five-step cleaning procedure applied to every raw light curve, executed in parallel with joblib.

1. Normalisation: the flux is divided by its median so that all stars share a unit baseline.
2. Outlier clipping: a 3-sigma iterative clip with up to five iterations removes cosmic-ray hits and flares.
3. Gap filling: gaps of up to three cadences are linearly interpolated, longer gaps are flagged and left open.
4. Detrending: a Wotan biweight sliding filter with a 0.75-day window removes slow stellar and instrumental trends.
5. Quality gate: light curves with too little baseline or too much scatter are rejected and reported.



**Figure 6.1: Effect of biweight detrending on a light curve with three transits (illustrative)**

### ***6.2.1 Choice of the detrending window***

The biweight window must be wide enough not to absorb a transit and narrow enough to follow stellar variability. A common rule of thumb is a window about three times the longest expected transit duration. Transit durations for the periods considered range from about one to six hours, so a window of 0.75 days (18 hours) protects the transit while following variations on timescales of a day or longer.

### ***6.2.2 Deviation: upper-only sigma clipping***

The specification called for symmetric sigma clipping. Testing showed this is incorrect when clipping is applied to un-detrended flux: a transit is a downward excursion, and a symmetric clip removes the very points that constitute the transit, both in synthetic and in real light curves. The implementation therefore clips only upward outliers. This single change restored the depth of injected transits (see Table 7.2).

### ***6.2.3 Deviation: robust quality gate***

The original quality gate rejected a light curve if its root-mean-square scatter exceeded five times an expected photon-noise figure computed from the number of cadences. Because this proxy depended only on the cadence count and ignored stellar brightness, it rejected about forty percent of all targets, and disproportionately the planet class (47 percent against 25 percent for non-planets), thereby biasing the dataset against exactly the class of interest. The gate was replaced by a robust point-to-point scatter measure based on the median absolute deviation with a ceiling of two percent, which rejects about ten percent of the targets and treats the classes evenly (Bug 8).

**Table 6.2: Preprocessing parameters**

| Constant               | Value      | Meaning                      |
|------------------------|------------|------------------------------|
| MIN_DAYS               | 13.5 days  | minimum usable baseline      |
| SIGMA_CLIP_SIG / ITERS | 3.0 / 5    | upper-only clip              |
| GAP_FILL_MAX_CAD       | 3 cadences | interpolate shorter gaps     |
| WOTAN_WINDOW           | 0.75 days  | biweight window length       |
| MAX_ROBUST_RMS         | 0.02       | robust scatter ceiling (2 %) |

### 6.3 Stage 3: Periodicity Detection

The detection module wraps the Transit Least Squares algorithm. TLS fits a limb-darkened transit template on a grid of periods, epochs and durations and computes the Signal Detection Efficiency, a normalised measure of how strongly the best period stands out from the periodogram background. The module bins the light curve to ten minutes for the search, splits stitched multi-sector data at gaps longer than thirty days, and extracts period, epoch, depth, duration, SDE, signal-to-noise ratio, false-alarm probability, radius ratio and transit count.

```
def detect_candidate(time, flat_flux, tic_id=""):
    res = run_tls(time, flat_flux)          # TLS on the best segment
    feats = extract_geometry_features(res, baseline)
    feats["is_candidate"] = (feats["SDE"] >= 7.0
                             and feats["FAP"] < 0.01)
    return feats
```

**Table 6.3: TLS configuration**

| Setting                    | Value                             | Note                                       |
|----------------------------|-----------------------------------|--------------------------------------------|
| Period range               | 0.5 to 27 days                    | up to one TESS sector                      |
| Oversampling factor        | 3                                 | specification: 5; reduced for laptop speed |
| Duration grid step         | 1.05                              |                                            |
| SDE for feature extraction | at least 5                        | keeps borderline signals                   |
| Candidate rule             | SDE at least 7 and FAP below 0.01 | strict triage                              |

### 6.3.1 Deviation: transit depth

TLS reports its depth attribute as the flux level at the bottom of the transit, not as the fractional dip. Using this value directly made every transit depth about 99 percent. The implementation computes one minus this value and prefers the radius-ratio output where available (Bug 1). A second, more subtle bug came from passing the keyword arguments `minimum_period` and `maximum_period`, which are not TLS parameters and were silently ignored, so that the search used its default range; the correct names are `period_min` and `period_max` (Bug 7).

## 6.4 Stage 4: Feature Engineering

The feature module turns each detrended light curve and its TLS output into a numerical vector of 65 to 73 columns. The features fall into nine groups that encode the physical questions an astronomer would ask when vetting a candidate.

**Table 6.4: Engineered feature groups**

| Group                  | Example features                                         | Physical question                           |
|------------------------|----------------------------------------------------------|---------------------------------------------|
| Transit geometry       | period, depth, duration_hr, transit_count, rp_rs         | How big, how long, how often?               |
| Signal quality         | SDE, SNR, FAP, phase_coverage                            | How significant is the signal?              |
| Morphology             | flat_bottom_score, ingress_egress_asym, phase_folded_std | Is the dip box-like or V-shaped?            |
| Odd/even               | odd_depth, even_depth, odd_even_ratio                    | Do alternate dips have equal depth?         |
| Secondary eclipse      | secondary_depth, secondary_ratio                         | Is there a dip at phase 0.5?                |
| Centroid               | centroid_proxy                                           | Does the image centre shift during the dip? |
| Stellar                | stellar_Teff,logg,rad,mass,dist,metallicity,tess_mag     | What kind of star is this?                  |
| Time-series statistics | RMS, skewness, kurtosis, autocorrelation                 | How noisy or variable is the star?          |
| Spectral (FFT)         | tsf_fft_coeff_k_real / abs on folded flux                | What periodic structure is in the shape?    |

#### **6.4.1 *Odd/even depth test***

If a signal is caused by an eclipsing binary at half the true period, the primary and secondary eclipses have different depths, so odd-numbered and even-numbered transits differ. For a real planet, both are equal within noise. The ratio of the two depths is therefore a strong discriminator, and it is computed with a fold of the light curve at the detected period.

#### **6.4.2 *Secondary eclipse and morphology***

The secondary-eclipse depth is measured at phase 0.5 relative to the primary. A flat-bottom score compares the depth at the centre of the transit with the depth near the ingress and egress. A planet crossing a stellar disc produces an approximately flat bottom, modified only by limb darkening, whereas a grazing stellar eclipse produces a V. The ingress/egress asymmetry measures the difference between the times taken to go into and come out of transit.

#### **6.4.3 *Centroid-shift proxy***

A background eclipsing binary that contaminates the aperture produces a dip that is displaced from the target. The proper test uses the flux-weighted centroid time series (MOM\_CENTR1 and MOM\_CENTR2 in SPOC files), and a commit at the end of the project added these columns. For light curves lacking centroid data, a proxy based on quality-flagged cadences during the transit is used.

#### **6.4.4 *GPU acceleration***

The heaviest numerical work is phase folding, time-series statistics and Fourier features. These are implemented twice in `gpu_features.py`: a CuPy and PyTorch version for NVIDIA GPUs, and an equivalent NumPy and

SciPy version. The module detects CUDA at import time, uses the GPU when present and falls back silently otherwise, so the same pipeline runs on a Mac laptop and on the RTX 4050 machine. The GPU cache is flushed every fifty files to prevent memory leaks.

#### 6.4.5 *Anti-hang engineering*

Early full-catalogue runs hung the workstation. Three protective measures were added. The worker count is capped at  $\min(\text{cpu\_count} \text{ minus } 2, 4)$ . A per-task timeout of ten minutes, implemented with threads to be Windows-compatible, kills a stuck TLS call. A psutil guard pauses task dispatch when free RAM falls below 1.5 GB. Results are checkpointed to disk every ten rows so that a crash loses at most ten targets of work.

**Table 6.5: Feature-extraction safeguards**

| Safeguard           | Setting                              |
|---------------------|--------------------------------------|
| Maximum workers     | $\min(\text{CPU count minus } 2, 4)$ |
| Per-task timeout    | 600 seconds                          |
| Minimum free RAM    | 1.5 GB                               |
| Checkpoint interval | every 10 rows                        |
| GPU cache flush     | every 50 files                       |
| Phase bins          | 200                                  |

### 6.5 Stage 5: Classification

The classifier (classifier.py) is a stacked ensemble with three layers. At the bottom, one or more gradient-boosted tree learners are trained. A HistGradientBoosting learner from scikit-learn is always available, with XGBoost and LightGBM added automatically if their OpenMP runtimes load. The middle layer is a logistic-regression meta-learner that takes the out-of-fold probabilities of the base learners as input. The top layer is a Platt-scaling logistic regression fitted on out-of-fold meta-learner scores to calibrate the final probability.

```

class StackedClassifier:
    """Base learners -> logistic meta-learner -> Platt calibration."""
    def fit(self, X, y, groups):
        # 1. out-of-fold probabilities from group-stratified CV
        # 2. meta-learner fitted on stacked OOF probabilities
        # 3. Platt calibrator fitted on OOF meta scores (nested)
    def predict_proba(self, X):
        mf = self._meta_features(X)
        return self.platt_.predict_proba(
            self.meta_.decision_function(mf).reshape(-1, 1))

```

**Table 6.6: Base-learner hyper-parameters (HistGradientBoosting)**

| Hyper-parameter   | Value                                        |
|-------------------|----------------------------------------------|
| learning_rate     | 0.05                                         |
| max_iter          | 300                                          |
| max_depth         | 6                                            |
| l2_regularization | 1.0                                          |
| early stopping    | validation fraction 0.15, 20 rounds patience |
| class_weight      | balanced (unless SMOTE is used)              |
| random_state      | 42                                           |

### 6.5.1 Why nested calibration

Fitting the meta-learner and calibrator on data the base learners had seen would make probabilities over-confident, so the pipeline fits the meta-learner on out-of-fold base predictions and the Platt calibrator on out-of-fold meta scores.

## 6.6 Leakage Control

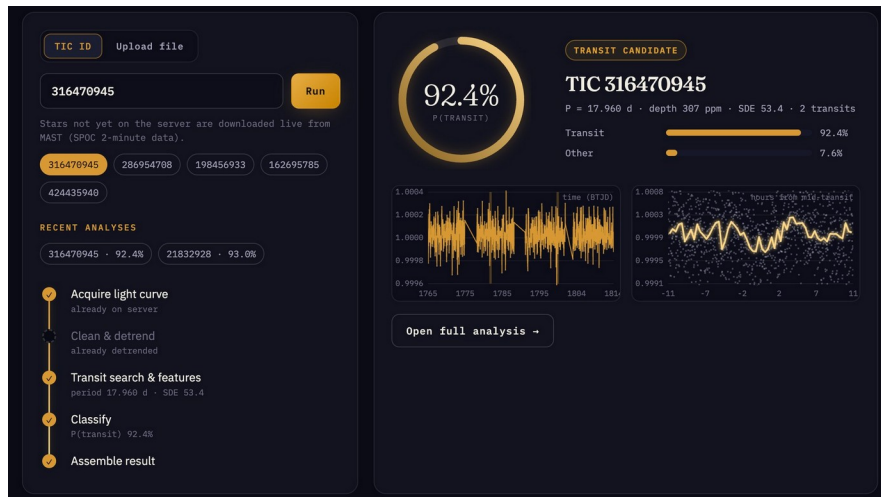
Leakage was the most instructive issue in the project; three controls were implemented.

- Group-aware splits: StratifiedGroupKFold by star keeps a real star and its synthetic clones together; synthetic rows are used for training only, and held-out evaluation is on real targets.
- Leaky columns removed: t0 and n\_cadences reflected how targets were collected, so both sit in LEAKY\_FEATURES and are never used.

## 6.7 Web Dashboard and API

The pipeline is exposed to researchers through a small web application. A FastAPI service (api/main.py) wraps the pipeline as asynchronous jobs held in a SQLite job store, and streams the progress of each stage to the browser as server-sent events. The front end is a set of static HTML, CSS and JavaScript pages served by the same process, so the whole application starts with one command: `uvicorn api.main:app`. The screenshots in this section were captured from the running application.

The Analyze panel (Figure 6.2) accepts a TIC identifier or an uploaded light curve, shows each stage as it completes and returns the calibrated score with the detrended light curve and the phase fold. The candidate explorer (Figure 6.3) ranks the real TESS stars by model score and can be filtered by catalogue label and minimum score, and a target page (Figure 6.4) gives the full light curve, phase fold and measured features of one star.



**Figure 6.2: Live analysis of TIC 316470945 with stage progress and calibrated score**

Search TIC...

All catalog labels

All stars

min P(transit) 0%

| TIC       | CATALOG | MODEL SAM IT? | MODEL SCORE | PERIOD (D) | DEPTH (PPM) | SDE  |
|-----------|---------|---------------|-------------|------------|-------------|------|
| 21832928  | Transit | split unknown | 93.8%       | 16.351     | 244         | 19.0 |
| 423618139 | Transit | split unknown | 92.9%       | 8.916      | 9791        | 4.7  |
| 22972926  | Transit | split unknown | 92.9%       | 24.707     | 121         | 11.4 |
| 121123361 | Transit | split unknown | 92.8%       | 5.265      | 960         | 6.2  |
| 293460551 | Transit | split unknown | 92.8%       | 10.100     | 6684        | 6.6  |
| 458494083 | Transit | split unknown | 92.8%       | 3.124      | 45          | 5.2  |
| 233435627 | Transit | split unknown | 92.8%       | 5.727      | 141         | 5.2  |
| 159716174 | Transit | split unknown | 92.8%       | 14.534     | 8883        | 6.7  |

Figure 6.3: Candidate explorer ranked by model score



Figure 6.4: Target page with light curve, phase fold and measured features

## 6.8 Source Code Overview

This section reproduces the core routine of each implemented stage. Docstrings and comment lines have been removed for brevity; the logic is unchanged.

### Listing 6.1: Acquisition: retry with exponential back-off (acquisition.py)

```
def _retry_call(fn, *args, max_retries: int = 3, base_delay: float = 2.0,
               max_delay: float = 30.0, what: str = "MAST call", **kwargs):
    last_exc = None
    for attempt in range(1, max_retries + 1):
        try:
            return fn(*args, **kwargs)
        except Exception as e:
            last_exc = e
            if attempt == max_retries:
                break
            msg = str(e).lower()
            is_rate_limited = "429" in msg or "too many requests" in msg
            delay = min(max_delay, base_delay * (2 ** (attempt - 1)))
            if is_rate_limited:
                delay *= 3
            delay += random.uniform(0, delay * 0.25)
            log.warning(" %s failed (attempt %d/%d): %s - retrying in %.1fs",
                      what, attempt, max_retries, e, delay)
            time.sleep(delay)
    raise last_exc
```

### Listing 6.2: Preprocessing: upper-only clipping, biweight detrending, quality gate (preprocessing.py)

```
def sigma_clip_flux(
    time: np.ndarray,
    flux: np.ndarray,
    sigma: float = SIGMA_CLIP_SIG,
    maxiters: int = SIGMA_CLIP_ITERS,
) -> tuple[np.ndarray, np.ndarray, np.ndarray]:
    from astropy.stats import sigma_clip as astropy_sigma_clip
    clipped = astropy_sigma_clip(flux, sigma_lower=np.inf, sigma_upper=sigma,
                                maxiters=maxiters, masked=True)
    good = ~clipped.mask & np.isfinite(flux) & np.isfinite(time)
    return time[good], flux[good], good

def detrend_flux(
    time: np.ndarray,
    flux: np.ndarray,
    method: str = "biweight",
    window_length: float = WOTAN_WINDOW,
) -> tuple[np.ndarray, np.ndarray]:
    from wotan import flatten
    finite = np.isfinite(time) & np.isfinite(flux)
    t_in = time[finite]
    f_in = flux[finite]
    if len(t_in) < 100:
        raise ValueError(f"Too few finite cadences for detrending ({len(t_in)}).")
    flat_flux, trend = flatten(
        t_in, f_in,
        method=method,
        window_length=window_length,
        return_trend=True,
        robust=True,
    )
    flat_full = np.full(len(time), np.nan)
    trend_full = np.full(len(time), np.nan)
    flat_full[finite] = flat_flux
    trend_full[finite] = trend
    return flat_full, trend_full
```

```

def quality_gate(
    time: np.ndarray,
    flat_flux: np.ndarray,
    tic_id: str,
) -> tuple[bool, str]:
    finite = np.isfinite(time) & np.isfinite(flat_flux)
    t_ok = time[finite]
    f_ok = flat_flux[finite]
    baseline = t_ok[-1] - t_ok[0] if len(t_ok) > 1 else 0.0
    if baseline < MIN_DAYS:
        return False, f"baseline {baseline:.1f}d < {MIN_DAYS}d"
    robust_rms = 1.4826 * float(np.nanmedian(np.abs(f_ok - np.nanmedian(f_ok))))
    if robust_rms > MAX_ROBUST_RMS:
        return False, f"robust rms {robust_rms:.4f} > {MAX_ROBUST_RMS}"
    return True, "ok"

```

**Listing 6.3: Detection: TLS period search and candidate decision (detection.py)**

```

def run_tls(time: np.ndarray, flat_flux: np.ndarray) -> dict:
    from transitleastsquares import transitleastsquares as TLS
    finite = np.isfinite(time) & np.isfinite(flat_flux)
    t = time[finite]
    f = flat_flux[finite]
    if len(t) < 200:
        return {}
    t, f = _bin_lightcurve(t, f)
    if len(t) < 100:
        return {}
    try:
        model = TLS(t, f)
        results = model.power(
            period_min = TLS_PERIOD_MIN,
            period_max = min(TLS_PERIOD_MAX, (t[-1] - t[0]) / 2),
            oversampling_factor = TLS_OVERSAMPLING,
            duration_grid_step = TLS_DURATION_STEP,
            use_threads = 2,
            show_progress_bar = False,
        )
        return results
    except Exception as e:
        log.debug(" TLS failed: %s", e)
        return {}

def detect_candidate(time: np.ndarray, flat_flux: np.ndarray, tic_id: str = "") -> dict:
    finite = np.isfinite(time) & np.isfinite(flat_flux)
    baseline = float(time[finite][-1] - time[finite][0]) if finite.sum() > 1 else 0.0
    feats = extract_geometry_features(run_tls(time, flat_flux), baseline)
    sde, fap = feats["SDE"], feats["FAP"]
    feats["is_candidate"] = bool(np.isfinite(sde) and sde >= SDE_MIN
                                and (not np.isfinite(fap) or fap < FAP_MAX))
    return {"tic_id": tic_id, **feats}

```

**Listing 6.4: Classification: base learners and stacked ensemble with Platt calibration (classifier.py)**

```

def make_base_learners(use_smote: bool = False, n_jobs: int = 2) -> dict:
    learners = {
        "hgb": HistGradientBoostingClassifier(
            learning_rate=0.05, max_iter=300, max_depth=6, l2_regularization=1.0,
            early_stopping=True, validation_fraction=0.15, n_iter_no_change=20,
            class_weight=None if use_smote else "balanced", random_state=SEED),
    }
    try:
        import xgboost as xgb
        learners["xgb"] = xgb.XGBClassifier(
            n_estimators=500, max_depth=6, learning_rate=0.05, subsample=0.8,
            colsample_bytree=0.8, eval_metric="logloss", n_jobs=n_jobs, random_state=SEED,
            tree_method="hist", device=xgb_device())
    except Exception as e:
        _warn_once("XGBoost unavailable – install the OpenMP runtime: `brew install libomp`.")
    try:
        import lightgbm as lgb
        learners["lgbm"] = lgb.LGBMClassifier(
            n_estimators=500, learning_rate=0.05, num_leaves=63, subsample=0.8,
            colsample_bytree=0.8, n_jobs=n_jobs, random_state=SEED, verbose=-1,

```

```

        class_weight=None if use_smote else "balanced")
except Exception as e:
    _warn_once("LightGBM unavailable – install the OpenMP runtime: `brew install libomp`.")
if use_smote:
    from imblearn.over_sampling import SMOTE
    from imblearn.pipeline import Pipeline
    from sklearn.impute import SimpleImputer
    learners = {k: Pipeline([("impute", SimpleImputer(strategy="median")),
                              ("smote", SMOTE(random_state=SEED)),
                              ("clf", v)]) for k, v in learners.items()}

return learners

class StackedClassifier:
    def __init__(self, use_smote: bool = False, n_splits: int = 5, n_jobs: int = 2):
        self.use_smote, self.n_splits, self.n_jobs = use_smote, n_splits, n_jobs
    def _oof(self, X, y, groups, make_model):
        """Out-of-fold class probabilities from group-stratified CV."""
        oof = np.zeros((len(y), len(self.classes_)))
        cv = StratifiedGroupKFold(self.n_splits, shuffle=True, random_state=SEED)
        for tr, va in cv.split(X, y, groups):
            m = make_model().fit(X.iloc[tr], y[tr])
            oof[va] = m.predict_proba(X.iloc[va])
        return oof
    def fit(self, X: pd.DataFrame, y: np.ndarray, groups: np.ndarray):
        self.classes_ = np.unique(y)
        y = np.searchsorted(self.classes_, y)
        self.feature_names_ = list(X.columns)
        proto = make_base_learners(self.use_smote, self.n_jobs)
        self.names_ = list(proto)
        oof_cols = []
        for name in self.names_:
            oof = self._oof(X, y, groups, lambda n=name: clone(make_base_learners(
                self.use_smote, self.n_jobs)[n]))
            oof_cols.append(oof if len(self.classes_) > 2 else oof[:, [1]])
        meta_X = np.hstack(oof_cols)
        self.meta_ = LogisticRegression(max_iter=1000).fit(meta_X, y)
        self.base_ = {n: clone(proto[n]).fit(X, y) for n in self.names_}
        self.platt_ = None
        if len(self.classes_) == 2:
            cv = StratifiedGroupKFold(self.n_splits, shuffle=True, random_state=SEED + 1)
            meta_oof = np.zeros(len(y))
            for tr, va in cv.split(meta_X, y, groups):
                m = LogisticRegression(max_iter=1000).fit(meta_X[tr], y[tr])
                meta_oof[va] = m.decision_function(meta_X[va])
            self.platt_ = LogisticRegression().fit(meta_oof.reshape(-1, 1), y)
        return self
    def _meta_features(self, X):
        cols = [self.base_[n].predict_proba(X[self.feature_names_]) for n in self.names_]
        return np.hstack([c if len(self.classes_) > 2 else c[:, [1]] for c in cols])
    def predict_proba(self, X: pd.DataFrame) -> np.ndarray:
        mf = self._meta_features(X)
        if self.platt_ is not None:
            return self.platt_.predict_proba(self.meta_.decision_function(mf).reshape(-1, 1))
        return self.meta_.predict_proba(mf)
    def predict(self, X):
        return self.classes_[np.argmax(self.predict_proba(X), axis=1)]

```

# CHAPTER 7

## TESTING AND VALIDATION

### 7.1 Testing Strategy

Two levels of testing are used across the pipeline. Unit-level tests (specified in this chapter, automation pending) check a single module in isolation with controlled input, and an integration and benchmark protocol checks the output of the whole pipeline against known astronomical ground truth. In addition, the project maintains a bug log in which every defect found during real-data testing is recorded with its root cause and fix, since real archive data exposed problems that no synthetic unit test had anticipated.

### 7.2 Unit Testing

**Table 7.1: Unit test cases**

| ID    | Module        | Test                                              | Expected result                           | Status  |
|-------|---------------|---------------------------------------------------|-------------------------------------------|---------|
| UT-01 | acquisition   | QUALITY bitmask applied to a mocked FITS table    | Cadences with bits 1,2,4,8,16,512 removed | Planned |
| UT-02 | acquisition   | Disposition mapping of PC, KP, CP, EB, BEB, FP, V | Labels 0,0,0,1,2,3,3                      | Planned |
| UT-03 | acquisition   | ID normalisation of TIC_123 and 123               | Both map to the same key                  | Planned |
| UT-04 | acquisition   | Resume run with existing NPZ files                | No re-download                            | Planned |
| UT-05 | preprocessing | Normalise a flux array                            | Median equals 1.0                         | Planned |
| UT-06 | preprocessing | Upper-only clip on data with a transit            | Transit points retained                   | Planned |
| UT-07 | preprocessing | Detrend an injected sinusoid                      | Residual RMS reduced by at least 90 %     | Planned |
| UT-08 | preprocessing | Gap of 3 vs 4 cadences                            | 3 filled, 4 flagged                       | Planned |
| UT-09 | detection     | TLS on injected 5-day transit                     | Period within 0.1 %                       | Planned |
| UT-10 | detection     | depth conversion                                  | 1 minus TLS flux level                    | Planned |

| ID    | Module     | Test                                  | Expected result             | Status  |
|-------|------------|---------------------------------------|-----------------------------|---------|
| UT-11 | features   | Determinism on identical input        | Identical feature vectors   | Planned |
| UT-12 | features   | NaN handling for missing stellar data | No exception; NaN preserved | Planned |
| UT-13 | features   | GPU vs CPU phase fold                 | Equal within 1e-6           | Planned |
| UT-14 | classifier | Probabilities sum to one              | Row sums equal 1            | Planned |
| UT-15 | classifier | Group split                           | No group on both sides      | Pass    |

*Note: Table 7.1 is the unit-test plan. The repository does not yet contain an automated test suite, so the status of every case is Planned; the behaviours were so far verified by interactive checks and by the real-data bug hunt described below.*

### 7.3 Integration Testing

The integration protocol runs the full pipeline against TESS Sector 1 and checks three acceptance criteria defined in the project requirements: recovery of at least eighty percent of confirmed TOIs at SDE of at least seven, a false-positive rate below two percent on background non-variable stars, and a full-sector runtime of no more than ninety minutes on an eight-core CPU. At the time of writing the first four stages have been exercised at the scale of several thousand light curves, and the classifier has been evaluated with group-aware cross-validation. The remaining acceptance checks are defined over the completed pipeline and are scheduled for when parameter fitting and reporting are in place.

### 7.4 Injection–Recovery Validation

An injection–recovery test inserts transits with known parameters into real light curves, runs the pipeline and measures how often the period is recovered. This test found several of the most important bugs. Before the injector and the clipping stage were corrected, none of sixteen injected signals was recovered within five percent. After the fixes, six of eight

injections were recovered in a partial run of the regenerated set. The full sensitivity map is future work (Section 10.4.6).

## 7.5 Bug Log Summary

Table 7.2 lists the root causes of the nine defects found and fixed during real-data testing.

**Table 7.2: Defects found during testing**

| # | Defect                       | Effect                                | Fix                                          |
|---|------------------------------|---------------------------------------|----------------------------------------------|
| 1 | TLS depth is a flux level    | all depths about 99 %                 | use 1 minus depth; prefer rp_rs              |
| 2 | CSV append with schema drift | duplicated or misaligned rows         | write mode on first write; canonical columns |
| 3 | ID format mismatch           | real labels almost all -1             | normalise IDs                                |
| 4 | Non-transiting injections    | identical copies labelled as transits | rejection sampling with batman               |
| 5 | Undetectable injections      | mislabelled positives                 | cap period; require SNR at least 8           |
| 6 | Symmetric sigma clip         | transits deleted                      | upper-only clip                              |
| 7 | Wrong TLS keyword names      | default range used                    | period_min / period_max                      |
| 8 | Photon-noise quality gate    | 40 % rejected, class bias             | robust scatter ceiling                       |
| 9 | Hung connections             | no progress for 10 min                | restart; timeout planned                     |

## 7.6 Evaluation Protocol for the Classifier

- Data: 2,271 labelled real targets, split 1,741 train and 530 test with StratifiedGroupKFold logic.
- Metrics: accuracy, macro-F1, per-class precision and recall, ROC-AUC, PR-AUC and Brier score, since the test set is imbalanced (356 Transit against 174 Other).
- Calibration and explanation: Platt scaling (Brier score), permutation importance and a learning curve.

## CHAPTER 8

### RESULTS AND DISCUSSION

*Reading guide. All results in this chapter are taken from the project's saved artefacts (models/metrics.json, preprocessing and manifest logs) and are measured, not illustrative.*

#### 8.1 Experimental Setup

**Table 8.1: Experimental setup**

| Item               | Configuration                                                  |
|--------------------|----------------------------------------------------------------|
| Data               | TESS Sector 1, 2-minute SPOC light curves, label-driven sample |
| CPU                | 8-core laptop CPU (development)                                |
| GPU                | NVIDIA RTX 4050 Laptop GPU, CUDA 12.9 (feature extraction)     |
| Software           | Python, Wotan, TLS, scikit-learn, CuPy, PyTorch                |
| Training rows      | 1,741                                                          |
| Held-out test rows | 530 (356 Transit, 174 Other)                                   |
| Number of features | 65                                                             |
| Random seed        | 42                                                             |

#### 8.2 Acquisition and Catalogue Statistics (REAL)

The acquisition run produced 2,360 downloaded and parsed TESS light curves, of which 2,120 were usable, comprising 1,423 transit-class and 697 other-class targets, together with 199 synthetic injections of which 192 were usable. The unified label catalogue contains 10,964 unique targets and the TIC stellar-parameter table 501 targets. The download was stopped at about 3,200 of 5,000 requested targets because of network-limited throughput and is fully resumable. About twenty-eight percent of failures were targets that have only full-frame-image data and no SPOC two-minute product, which this pipeline cannot process.

**Table 8.2: Pipeline data volumes by stage (REAL)**

| Stage                       | Count  | Notes                                      |
|-----------------------------|--------|--------------------------------------------|
| Downloaded light curves     | 2,360  | TESS Sector 1, PDCSAP flux                 |
| Rejected by quality gate    | 247    | 10.5 % of downloads (earlier gate version) |
| Successfully detrended      | 2,312  | median baseline 54 days                    |
| Unified label catalogue     | 10,964 | targets from three catalogues              |
| Stellar cross-matched (TIC) | 501    | Teff, log g, radius, mass, distance        |
| Classifier training rows    | 1,741  | group-aware split                          |
| Classifier held-out rows    | 530    | no star overlap with training              |

### 8.3 Preprocessing Performance

Across the 2,312 detrended light curves, the biweight filter reduced flux RMS by a mean of 45.6 percent and a median of 49.4 percent relative to the raw normalised flux, with a range from minus 3.6 percent to 99.3 percent. A few already-quiet targets show a marginal increase after detrending, which is expected when there is essentially no long-term trend to remove. As a representative example, target TIC 144065872, with 19,226 cadences and a 27.9-day baseline, had its RMS reduced from 0.0024 to 0.0008, a 68 percent reduction, while retaining its transit-like dips.

### 8.4 Classification Performance (REAL)

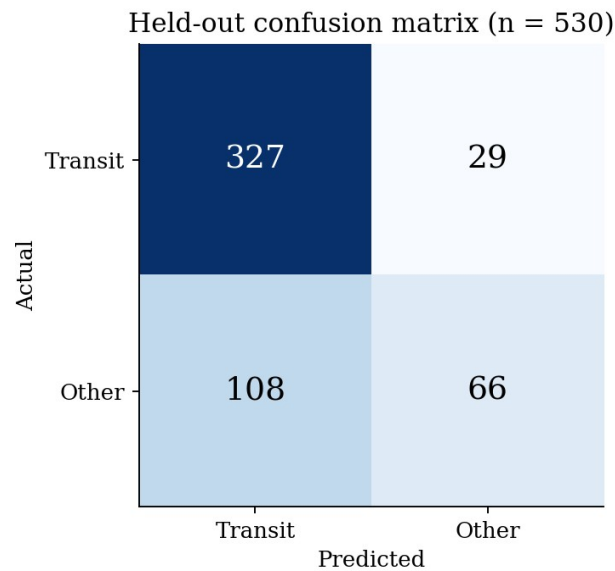
The stacked classifier was evaluated on the held-out test set of 530 real targets. It reached an accuracy of 74.2 percent, a macro-F1 of 0.659, a ROC-AUC of 0.750, a PR-AUC of 0.840 and a Brier score of 0.182. Table 8.3 lists these values against the project targets, and Table 8.4 gives the per-class breakdown.

**Table 8.3: Classifier metrics against project targets (REAL)**

| Metric             | Held-out value | Project target  |
|--------------------|----------------|-----------------|
| Accuracy           | 74.2 %         | not specified   |
| Macro-F1           | 0.659          | 0.90 or higher  |
| ROC-AUC            | 0.750          | not specified   |
| PR-AUC             | 0.840          | not specified   |
| Brier score        | 0.182          | lower is better |
| Transit recall     | 91.9 %         | not specified   |
| Other-class recall | 37.9 %         | not specified   |

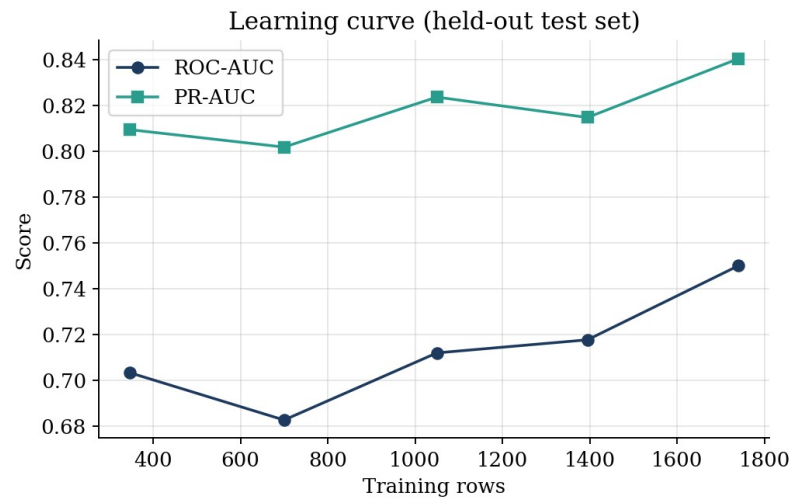
**Table 8.4: Per-class results (REAL)**

| Class            | Precision | Recall | F1    | Support |
|------------------|-----------|--------|-------|---------|
| Transit (0)      | 0.752     | 0.919  | 0.827 | 356     |
| Other (3)        | 0.695     | 0.379  | 0.491 | 174     |
| Macro average    | 0.723     | 0.649  | 0.659 | 530     |
| Weighted average | 0.733     | 0.742  | 0.716 | 530     |

**Figure 8.1: Held-out confusion matrix (REAL)**

The confusion matrix in Figure 8.1 shows that 327 of 356 transit targets are correctly recovered and only 29 are missed, while 108 of the 174 Other-class targets are wrongly accepted as transits. The classifier is therefore high in recall but low in specificity. This is the expected signature of a

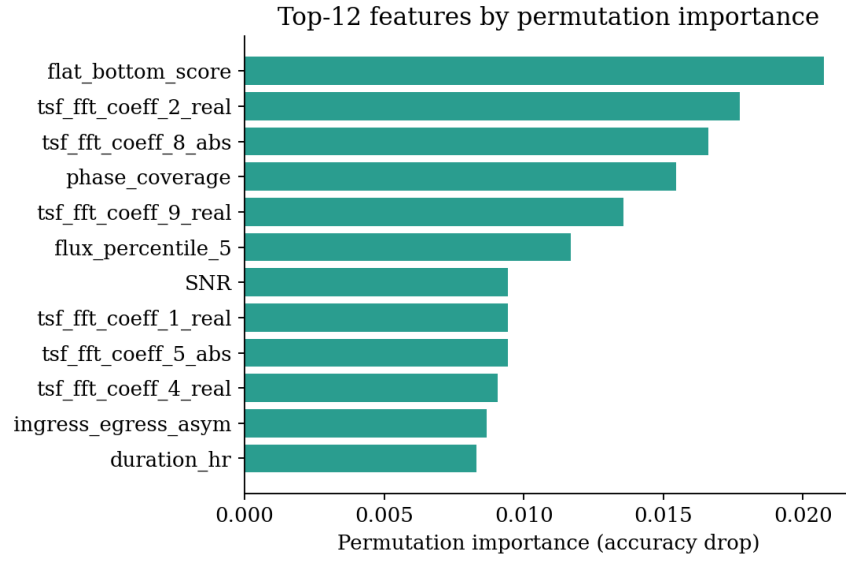
moderately imbalanced binary problem trained on features that discriminate weakly between a planet and the class of false positives that mimic one.



**Figure 8.2: Learning curve on the held-out test set (REAL)**

Figure 8.2 shows that ROC-AUC rises from 0.703 with 346 training rows to 0.750 with the full 1,741 rows, with some non-monotonic fluctuation at intermediate sizes because each point is a single fit. PR-AUC follows a similar pattern from 0.809 to 0.840. The curve has not fully flattened, which suggests that further labelled data would help, and that the model is not yet limited purely by the capacity of the feature set.

### 8.4.1 Feature importance



**Figure 8.3: Top-12 features by permutation importance (REAL)**

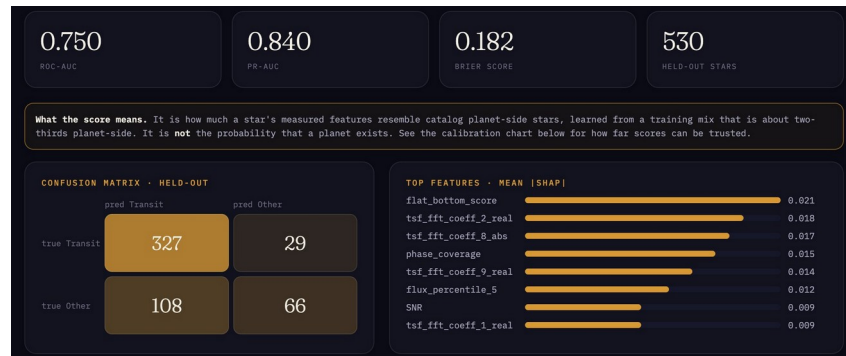
Permutation importance on the held-out fold, shown in Figure 8.3, ranks `flat_bottom_score` first, followed by several Fourier coefficients of the phase-folded flux, `phase_coverage`, the fifth-percentile flux, the signal-to-noise ratio, ingress/egress asymmetry and transit duration. These are physically meaningful transit-shape and signal-quality features rather than incidental statistics, which supports the interpretability goal of the design. The absolute importances are small (at most about 0.02) because many correlated features share the predictive information, so shuffling any single one costs little.

**Table 8.5: Top-10 permutation importances (REAL)**

| Rank | Feature              | Importance |
|------|----------------------|------------|
| 1    | flat_bottom_score    | 0.0208     |
| 2    | tsf_fft_coeff_2_real | 0.0177     |
| 3    | tsf_fft_coeff_8_abs  | 0.0166     |
| 4    | phase_coverage       | 0.0155     |
| 5    | tsf_fft_coeff_9_real | 0.0136     |
| 6    | flux_percentile_5    | 0.0117     |
| 7    | SNR                  | 0.0094     |

| Rank | Feature              | Importance |
|------|----------------------|------------|
| 8    | tsf_fft_coeff_1_real | 0.0094     |
| 9    | tsf_fft_coeff_5_abs  | 0.0094     |
| 10   | tsf_fft_coeff_4_real | 0.0091     |

The dashboard presents the same held-out evaluation in its model-performance view (Figure 8.4): the metric tiles, the confusion matrix and the SHAP feature ranking. The values match Tables 8.3 and 8.4.



**Figure 8.4: Model performance view of the dashboard**

## 8.5 Discussion

The results show that the first five stages of the pipeline run at the scale of thousands of light curves without manual intervention between stages, and that a first classifier trained on physically meaningful features reaches a ROC-AUC of 0.75. They also show that the classifier is currently far from the project target of macro-F1 at least 0.90. Three factors explain most of the gap. First, the class Other is heterogeneous: it mixes eclipsing binaries, blends and variable stars, which occupy different regions of feature space, and a binary framing forces the classifier to draw one boundary between transits and all of them. Second, several features that would separate planets from binaries most cleanly, notably true centroid motion and high-resolution secondary-eclipse search, have only just been added or exist as proxies. Third, the training set is small, at 1,741 rows.

## CHAPTER 9

### COMPARATIVE ANALYSIS AND IMPACT

#### 9.1 Comparison of Development Strategies

The first strategy is faster to a first result, since each script can be written and tested independently, but every hand-off between scripts is a place where format drift, silent truncation or an unlogged manual filter can enter the analysis unseen. The second strategy costs more up front, since a shared message format, an orchestrator and quality gates have to be designed first, but each hand-off becomes a logged, typed message and the same quality gating and resumability apply whether the run covers one target or a whole sector.

**Table 9.1: Hand-chained scripts versus orchestrated pipeline**

| Aspect               | Hand-chained scripts      | LuNaVision AI pipeline                     |
|----------------------|---------------------------|--------------------------------------------|
| Time to first result | Fast                      | Slower (interfaces first)                  |
| Reproducibility      | Depends on notes          | Manifests, logged messages                 |
| Resumability         | Usually restart           | Skip-if-exists, checkpoints                |
| Failure isolation    | Errors propagate silently | Stage boundaries; retries                  |
| Scaling              | Manual per script         | Independent per stage; GPU placement       |
| Auditability         | Low                       | Every decision reconstructible             |
| Leakage control      | Rarely addressed          | Group-aware splits; leaky features removed |

#### 9.2 Societal and Educational Impact

Automated screening does not replace the astronomer, but it shifts effort from routine elimination of false positives to the ambiguous cases. Lower barriers to access could allow smaller institutions to take part in candidate vetting, and the dashboard of Section 6.7 can be used in teaching to show why a flat-bottomed dip is planetary while a V-shaped one is stellar.

# CHAPTER 10

## RISKS, CONCLUSION AND FUTURE ENHANCEMENT

### 10.1 Risks and Mitigations

**Table 10.1: Identified risks and mitigations**

| Risk                               | Impact                              | Mitigation                                                                        |
|------------------------------------|-------------------------------------|-----------------------------------------------------------------------------------|
| MAST bandwidth and rate limits     | Incomplete sector download          | Local caching; resumable thread-pooled downloads; back-off with jitter; manifests |
| Severe class imbalance             | Classifier biased to majority class | Class weights; SMOTE option; synthetic injection                                  |
| Over- or under-detrending          | Missed detections                   | Biweight window tuned to transit timescale; injection-recovery tests              |
| MCMC non-convergence               | Unreliable uncertainties            | Bounded iterations; least-squares fallback                                        |
| Unrecognised morphologies          | Misclassification of novel signals  | Confidence routing to human review                                                |
| Data leakage                       | Inflated metrics                    | Group-aware splits; leaky columns removed; real-only test set                     |
| Hung tasks and memory exhaustion   | Stalled runs                        | Timeouts; RAM guard; worker caps; checkpoints                                     |
| FFI-only targets without SPOC data | About 28 % of downloads fail        | Out of scope; future support for full-frame images                                |

### 10.2 Limitations

- The classifier is binary, since eclipsing-binary and blend labels are absent from the current light-curve sample.
- Macro-F1 of 0.659 is below the 0.90 target; the Other class has low recall of 37.9 percent.
- The training set contains 1,741 rows from one sector, which limits generalisation to other sectors and cadences.
- Parameter fitting, reporting and active learning are not yet implemented.

- The centroid feature is only a proxy for light curves without centroid columns.
- The repository does not yet contain an automated test suite.

### 10.3 Conclusion

This report presented LuNaVision AI, a multi-agent pipeline for automated exoplanet discovery from TESS observations, and described the portion of the system that has been implemented and evaluated. The data-acquisition, preprocessing, periodicity-detection, feature-engineering and classification stages are functional and have been run on thousands of real light curves: 2,360 TESS light curves were acquired, 2,312 were detrended with a median noise reduction of about 49 percent, and a 65-feature representation was computed for each candidate. An initial stacked classifier, evaluated with group-aware splitting on 530 held-out real targets, reaches 74.2 percent accuracy, a ROC-AUC of 0.750 and a PR-AUC of 0.840, and relies mainly on physically meaningful transit-shape and signal-quality features.

Equally important are the engineering lessons. Nine defects, most of them silent, were found by checking pipeline output against physical expectations: a transit depth of ninety-nine percent, a sigma clip that deleted transits, a quality gate that discarded the planet class disproportionately and a timestamp column that leaked the label. The experience shows that the credibility of a machine-learning result in astronomy depends as much on the rigour of the surrounding pipeline as on the choice of classifier.

Taken together, the completed and planned stages support the central claim of the project: that exoplanet discovery from TESS light curves can be organised as a single coordinated multi-agent pipeline instead of a

sequence of disconnected, manually operated tools. The absolute performance has not yet reached the project targets, but the architecture, the evaluation protocol and the identified path to improvement are sound.

## **10.4 Future Enhancements**

### ***10.4.1 Orbital parameter estimation***

Implement the Parameter Fitting Agent. Initialise a batman model from each candidate's TLS parameters, run emcee with 32 walkers and 1,000 steps and report posteriors and 95 percent credible intervals for radius ratio, period, impact parameter, inclination and mid-transit time, with a least-squares fallback.

### ***10.4.2 Automated visualisation and reporting***

Implement the Reporting Agent to generate seven diagnostic plots per candidate: raw and detrended light curve, TLS periodogram, phase-folded transit with model overlay, MCMC corner plot, odd/even comparison and centroid-motion plot, together with a ranked catalogue and a multi-page PDF summary.

### ***10.4.3 Four-class taxonomy***

Extend the labelled dataset to populate the eclipsing-binary and blend classes and retrain the classifier as a four-class model, using the centroid time series (MOM\_CENTR1 and MOM\_CENTR2) as the key discriminator for background blends.

### ***10.4.4 Active learning loop***

Wire the classifier's low-confidence output into an active-learning agent, route those candidates to a human reviewer through the dashboard, and fold newly confirmed labels back into the next training run.

#### ***10.4.5 Deep-learning and explainability upgrades***

Add a one-dimensional CNN on the phase-folded flux as an additional base learner in the stack, and adopt SHAP attributions to explain individual decisions in addition to global permutation importance. Hyper-parameter optimisation with Optuna is already in the dependency list.

#### ***10.4.6 Validation at scale***

Run the injection–recovery sensitivity test on the full pipeline, measure recovery of confirmed TOIs at SDE of at least seven, and extend acquisition, preprocessing and detection to further TESS sectors, moving toward the 90-minute-per-sector runtime target.

#### ***10.4.7 Test automation***

Convert the unit-test plan of Chapter 7 into an automated suite run in continuous integration, with fixtures for mocked MAST responses and synthetic light curves.

#### ***10.4.8 Productisation***

Build the API, dashboard, authentication and orchestration layers of a multi-user service and pilot it with a small group of student astronomers.

## APPENDIX A: FEATURE DICTIONARY

Table A.1 lists the principal columns of the feature matrix, their meaning and unit. The matrix has 65 model features after identifiers, labels and leaky columns are excluded; the CSV itself holds up to 73 columns.

**Table A.1: Feature dictionary**

| Feature                    | Meaning                                 | Unit                    |
|----------------------------|-----------------------------------------|-------------------------|
| period                     | Detected orbital period from TLS        | days                    |
| depth / depth_ppm          | Fractional transit depth                | ratio / ppm             |
| duration_hr                | Transit duration                        | hours                   |
| transit_count              | Number of transits in the baseline      | count                   |
| SDE                        | Signal Detection Efficiency             | none                    |
| SNR                        | Signal-to-noise ratio of the transit    | none                    |
| FAP                        | False-alarm probability                 | none                    |
| rp_rs                      | Planet-to-star radius ratio             | none                    |
| phase_coverage             | Fraction of orbital phase with data     | none                    |
| flat_bottom_score          | Flatness of transit bottom (U versus V) | none                    |
| phase_folded_std           | Standard deviation of folded curve      | flux                    |
| ingress_egress_asym        | Asymmetry of ingress and egress         | none                    |
| secondary_depth            | Depth at phase 0.5                      | ratio                   |
| secondary_ratio            | Secondary to primary depth              | none                    |
| odd_depth / even_depth     | Depth of odd and even transits          | ratio                   |
| odd_even_ratio             | Ratio of odd to even depth              | none                    |
| centroid_proxy             | Proxy for centroid shift during transit | none                    |
| hz_proximity               | Proximity to habitable zone             | none                    |
| stellar_Teff               | Effective temperature                   | K                       |
| stellar_logg               | Surface gravity                         | log10 cm/s <sup>2</sup> |
| stellar_rad / mass         | Stellar radius and mass                 | solar units             |
| stellar_dist_pc            | Distance                                | parsec                  |
| stellar_metallicity        | [Fe/H]                                  | dex                     |
| tess_mag                   | TESS magnitude                          | mag                     |
| flux_percentile_5          | Fifth percentile of folded flux         | flux                    |
| above_3sigma_frac          | Fraction of points above 3 sigma        | fraction                |
| tsf_fft_coeff_k_real / abs | FFT coefficient k of folded flux        | none                    |

## APPENDIX B: COMMAND-LINE USAGE

The main stages are run from the command line inside the `exoplanet_pipeline` directory. The commands below illustrate typical use.

```
# 1. Acquire labelled targets (label-driven, at most 2 sectors each)
python src/acquisition.py --sector 1 --labeled-targets 5000 \
    --max-sectors 2 --concurrency 8

# 2. Preprocess: normalise, clip, gap-fill, detrend, quality-gate
python src/preprocessing.py --n-jobs -1 --window 0.75

# 3. Extract features (GPU used automatically if available)
python src/features.py

# 4. Train the stacked classifier
python src/classifier.py --features data/catalogs/feature_matrix.csv \
    --out models/ --learning-curve
```

## REFERENCES

- [1] B. Y. Feng, R. Ferrer-Chávez, A. Levis, W. T. Freeman, and J. J. Wang, "Exoplanet imaging via differentiable rendering," *Int. J. Data Sci. Anal.* (Springer), 2025.
- [2] M. Jara-Maldonado et al., "Light intensity of stars for exoplanet detection," in *Proc. IEEE Region 10 Symp. (TENSYP)*, 2022.
- [3] J. Pimentel, J. Amorim, and Rudzicz, "Feature extraction for exoplanet detection," *Int. J. Data Sci. Anal.* (Springer), 2024.
- [4] M. Ansdell et al., "Scientific domain knowledge improves exoplanet transit classification with deep learning," *Astrophys. J. Lett. (IOP/AAS)*, 2018.
- [5] I. Priyadarshini and V. Puri, "A CNN-based ensemble model for exoplanet detection," *Earth Sci. Inform.* (Springer), 2021.
- [6] Z. L. de Beurs et al., "Identifying exoplanets with deep learning IV: Removing stellar activity signals from radial velocity data using neural networks," *Astron. J. (IOP/AAS)*, 2022.
- [7] M. S. Khan, J. Jenkins, and N. B. Yoma, "Discovering new worlds: Signal processing methods for RV exoplanet detection," *IEEE Signal Process. Mag.*, vol. 34, pp. 104-115, 2017.
- [8] J. T. Wright and B. S. Gaudi, "Exoplanet detection methods," in *Planets, Stars and Stellar Systems*, vol. 3. Springer Netherlands, 2012.
- [9] M. Hippke and R. Heller, "Optimized transit detection algorithm to search for periodic transits of small planets," *Astron. Astrophys.*, 2019.
- [10] M. Hippke et al., "Wotan: Comprehensive time-series de-trending in Python," *Astron. J.*, 2019.
- [11] Lightkurve Collaboration, "Lightkurve: Kepler and TESS time series analysis in Python," *Astrophysics Source Code Library*, 2018.
- [12] G. R. Ricker et al., "Transiting Exoplanet Survey Satellite (TESS)," *J. Astron. Telesc. Instrum. Syst.*, vol. 1, no. 1, 014003, 2015.
- [13] L. Kreidberg, "batman: BAsic Transit Model cAlculationN in Python," *Publ. Astron. Soc. Pac.*, vol. 127, pp. 1161-1165, 2015.
- [14] D. Foreman-Mackey, D. W. Hogg, D. Lang, and J. Goodman, "emcee: The MCMC Hammer," *Publ. Astron. Soc. Pac.*, vol. 125, pp. 306-312, 2013.